

Can This Agent Even Do That?

Pre-Execution Validation for Agent Skills

MLADS+ Agents December 2026
Supplementary full paper

Abstract

Organizations need agents that complete requested work, not just produce plausible plans. SkillC checks whether a task can succeed with the declared tools, inputs and sequence of steps before the agent executes it. People review the task description; an optional language model can help prepare it, consuming model tokens, but logical rules determine the result. In 56 live-backend scenarios, checking first reduced model-token use by 46.74%, with all 28 achievable tasks completed both with and without the gate. A separate repeated-use study with simulated tools reduced tokens by 90.47%. On 490 real skill-runtime pairs, drawn from public skill repositories and executed blindly to obtain ground truth, verdicts produced from the skill text alone agreed with execution in 88% of decided cases, catching 84% of impossible pairs and wrongly blocking 10% of achievable ones; a direct model verdict was as accurate at lower cost, so the logical block is justified by its witness and re-checkability, not by accuracy. These comparisons count total model use, including any compaction; admitted agent execution can still consume tokens. We explain the workflow, implementation, evidence and limits in plain language, then provide the original mathematical definitions and complete proofs. SkillC complements authorization and runtime verification; it does not guarantee that a task description captures the user's intent or that real tools behave as described.

1 Introduction

Our business goals are faster delivery, lower model-token use, and greater client trust. Tokens are the units a model uses to process prompts and generate responses. Repeated attempts consume tokens even when an agent lacks something essential to succeed. For example, a calendar agent may be allowed to create an event but have no way to invite its participants. Permission to act does not make the requested outcome achievable.

SkillC addresses this gap by checking the task before execution. Its contribution is a working combination of checks for available operations, compatible steps and reachable goals, together with an explanation of each decision. The authors' applied work includes the task descriptions, checking method, implementation and comparative experiments. A language model can help write a description, but does not decide whether it passes.

This full paper supports the accompanying four-page Pre-Execution Validation for Agent Skills proposal. Sections 3–6 expand its method, data, results and implementation: Section 3.6 describes Controlled English and runtime-bound compaction, Section 4.1 and Sections 5.5–5.6 the real-skill benchmark with blind-execution ground truth, and Sections 6.2–6.3 the runtime monitor and the pre-session hook. Appendices A–D retain the definitions, mathematical results and proofs; Appendix D.8 states which result covers each verdict the implementation returns. Appendix E separates historical cost estimates

from current recomputation; Appendix F gives the supporting measurements. The main proposal remains the required submission template.

2 Related Work

In a usual agent workflow, the model chooses actions, observes tool responses and may retry after a failure. That is the baseline tested here: the same agent and task conditions without SkillC. We do not claim a measured advantage over every published agent system.

Automated planning studies which actions can lead to a goal [1]. Session types describe which messages and actions participants may exchange so that their work fits together [2, 3, 4]. Logical constraint solving checks whether stated conditions can hold together [5, 6]. SkillC brings these ideas together before agent execution.

ADAS and AFlow use models to construct or improve agent workflows [7, 8]. Research on multi-agent failures shows why coordination and task verification matter [9]. VeriGuard addresses policy safety, while SkillSpector screens skills for security concerns [10, 11]. These are related but different questions: generating a plan, enforcing a policy, or screening a skill does not by itself establish that the requested outcome is reachable. SkillC complements these approaches; it is not a replacement for them or a security boundary.

Translating natural-language tasks into planning formalisms has its own literature. LLM+P has a model write a PDDL problem and hands it to a classical planner [20]. Planetarium measures how often such translations are right: most model-written PDDL problems parse and are solvable, but only about a third match the intended meaning [21]. That gap is why SkillC treats compaction as untrusted and evaluates it against executed outcomes (Section 5.5) rather than against author-written labels.

3 Methodology

3.1 What the system checks

The input is a reviewed task description: the desired outcome, available operations, starting conditions and permitted sequence of steps. The following terms give short names to these ideas.

Term	Meaning in this paper
Skill	Instructions describing how an agent should carry out a type of work.
Capability	An available operation, together with when it may be used and what it is expected to change.
Contract	A reviewed description of the goal, available operations, required inputs and assumptions.
Guard / effect	A condition that must hold before an operation / the change it is expected to make.
Protocol	The allowed order of actions and messages between participants.
Compaction	Preparing a compact, structured description from longer instructions; model-assisted preparation consumes tokens.
Witness / blocking evidence	A possible sequence of steps / an explanation of why the checked task cannot succeed.
Static validation / runtime verification	Checking the description before execution / checking what actually happened during execution.

SkillC first asks whether the available operations can establish what the goal requires. It then checks whether the stated steps and participant interactions fit together. Finally, it explores the allowed steps while tracking the conditions and changes they describe. A logical solver checks those conditions; this is deterministic static validation, not another model’s opinion.

There are three results. **ACHIEVABLE** includes a possible route under the stated assumptions. **IMPOSSIBLE** includes a blocker and identifies what was ruled out. **UNKNOWN** means the available analysis does not decide the case. A failure of one particular workflow does not rule out every alternative: a goal-wide rejection needs evidence that the available operations cannot achieve the goal, independently of that workflow.

3.2 Prepare reusable facts, then check each request

Figure 1 connects preparation to use. First, describe what the available skills and tools can do, under what conditions, and within which policies. Review and store those compact facts. For a request, identify its goal, constraints and required outcomes; select the relevant facts; combine them into a task description; and check whether the requested outcome can be reached: first whether the tools and inputs exist (Check 1), then whether the steps can reach the goal (Check 2).

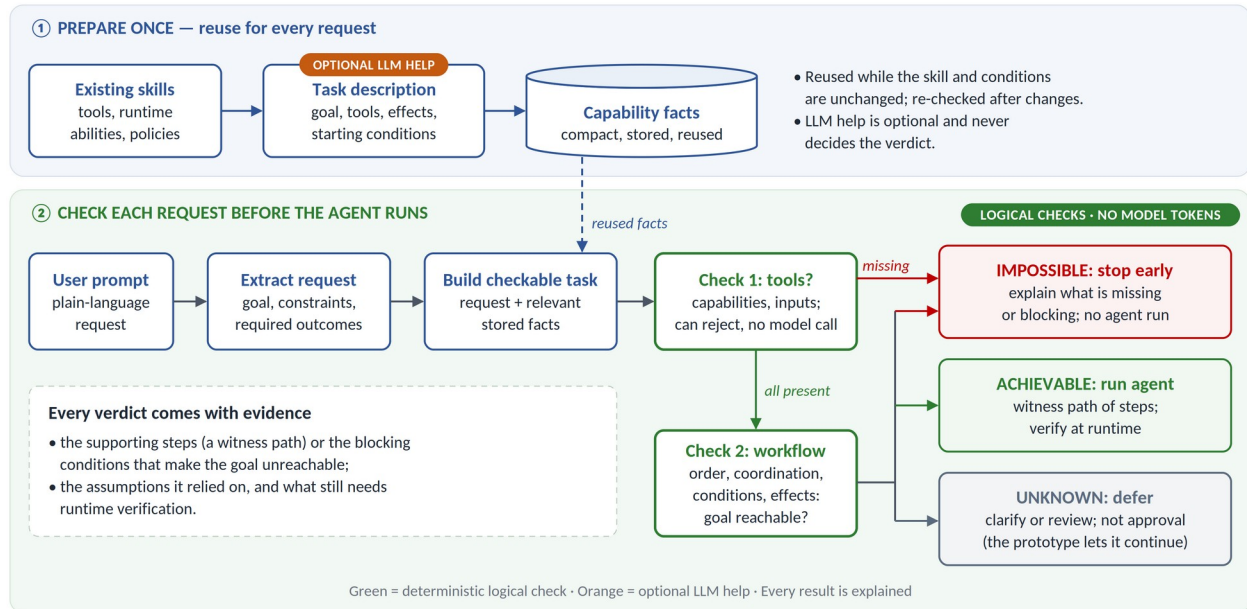


Figure 1. The two-stage workflow: prepare reusable capability facts once (top panel), then check each request before the agent runs (bottom panel). **UNKNOWN** means that the check has not established an answer; it is not evidence of success.

The workflow supports reusing preparation, not blindly reusing a decision for a different task. A changed goal, tool, input or policy can change the answer and requires another check. The measured reuse experiment held the task and its relevant conditions unchanged. It did not measure general search over a production-wide catalogue of reusable facts.

3.3 What the AI produces

Optional model assistance translates longer skill instructions into a structured description of goals, operations, conditions, effects and steps. That proposed description must be reviewed. The model does not issue the static verdict. After admission, the runtime agent may use a model to choose actions and interpret observations. Preparing a description can therefore use tokens even though the checking step

itself uses none. The live study used GPT-5.4 through Azure OpenAI. The real-skill benchmark (Section 4.1) used one Claude model family for compaction, blind execution and adjudication, invoked as sub-agents of the experiment harness; the direct-verdict baseline used the same family.

The reported experiments use an existing pretrained model, not a newly trained predictor. No model training, fine-tuning, training loss or learning curve is part of these studies. Training/validation/test partitions are consequently not applicable to the deterministic checker. A separate prompt-development study reused its evaluation scenarios after observing errors; it is development evidence, not a held-out generalization result.

3.4 Unexpected inputs and independent safeguards

Malformed descriptions are input errors, not successful checks or proof that a task is impossible. Unsupported cases and search limits must not be reported as established answers. Missing required information calls for clarification. Authorization and policy checks independently control which operations may run; runtime verification checks the actual results. Consequential actions need approval before execution because checking their results cannot undo a completed side effect.

3.5 How the theory supports the implementation

The reasoning develops from action-based planning and coordination rules to a model that tracks both the workflow and the conditions needed for its goal. A “state” is a snapshot of those conditions; a “transition” is one permitted action or message. The checker may deliberately include extra possible states so that it does not reject a real possibility merely because its model is simplified. This is an *over-approximation*. It can admit a route that will not work in reality, so admission is not a promise of success.

The key result says that, under the stated assumptions, finding no route even in that enlarged model rules out a route in the described system (Theorem 1). Theorems 4 and 5 connect the coordination rules to permitted steps. Theorem 6 states when the search terminates. Appendices A–C define the notation; Appendix D contains all six theorems, five lemmas, two propositions, one corollary and their proofs. A lemma is a supporting mathematical result; a proposition states a further result or limit; a corollary follows from the preceding results.

Selected results have proofs checked by Coq, a mathematical proof-checking tool. This is not an end-to-end proof of the application. The assumptions still require tools to have the declared effects and people to confirm that the description expresses the intended task.

3.6 Controlled English and runtime-bound compaction (P2g)

When a skill does not declare its tools, a model must state what the skill requires. In the original design the model wrote a JSON description. In the current design it writes SkillC Controlled English (CE): a small formal language in which every sentence form denotes exactly one construct of the checked description, so parsing is deterministic and exactly invertible, and a parse error names the line and column. Listing 1 is a complete skill.

```

Skill `book-flight`.
Tool `search_fares` (owner `agent`): via `browser`; adds `flight_selected`.
Tool `book_flight` (owner `agent`): via `bash`; needs `airline_account`;
    requires `flight_selected`; adds `booked`.
Goal: `booked`.
Protocol:
- `agent` uses `search_fares`.
- `agent` uses `book_flight`.

```

Listing 1. A skill in Controlled English. Bound to the developer sandbox, which grants no airline account, `book_flight` is blocked by a guard that never holds and the verdict is IMPOSSIBLE [BLOCKED_GUARD] naming that tool. Bound to the office-assistant runtime, which has no shell and no browser, both tools are withdrawn and the verdict is IMPOSSIBLE [MISSING_CAPABILITY]. Both verdicts were reproduced with the released checker.

A CE tool carries four optional clauses that bind it to a runtime: *via* (the runtime tool that performs it), *needs* (a resource from outside the runtime, such as an account or credential), *runs* (a program it depends on) and *effect* (local, `reads_external`, `writes_external` or `publishes`). An optional Live goal separates a deployed outcome from the core deliverable. Thinking steps are not tools, and optional work goes on a branch the agent may skip.

P2g, the procedure evaluated in Section 5.5, has six steps. Only steps 2 and 5 involve the model.

Step	Who	What happens
1. Runtime manifest	reviewed data	Lists the tools the runtime offers, the resources it grants and the effects it forbids. The developer sandbox offers <code>bash</code> , <code>read</code> , <code>write</code> , <code>edit</code> , <code>web_fetch</code> , <code>browser</code> and <code>ask_user</code> , grants no account or credential, and forbids <code>writes_external</code> and <code>publishes</code> .
2. Compaction	model	Writes the skill in CE with <i>via</i> and <i>needs</i> on every tool. If the reply does not parse, the model gets one retry with the located error.
3. Binding	deterministic	A tool whose <i>via</i> is absent from the manifest is withdrawn; a tool whose <i>needs</i> is not granted is blocked by a guard that never holds; branches of the agent's own choices that only a withdrawn tool could run are pruned.
4. Check	deterministic (Z3)	ACHIEVABLE, IMPOSSIBLE or UNKNOWN on the bound description, with witness or blocking frontier.
5. Repair	model	After IMPOSSIBLE, one repair guided by the counterexample. It may fix a compaction error; it may not add abilities the runtime lacks.
6. Guard	deterministic	The repair is rejected if it changes the Goal or Live goal, or if a kept tool loses a <i>needs</i> , <i>runs</i> or <i>effect</i> . A rejected repair leaves the refuted description in force.

Round trip and error coverage are tested: parsing the rendering of a description gives the description back, for every description in the repository, every mutant of those descriptions and property-generated descriptions with adversarial identifiers, and 24 malformed forms give located errors. The JSON path remains available. Section 5.5 shows that the surface language changes economy and well-formedness, not accuracy; the accuracy gain over the original JSON design comes from the modelling rules, the runtime binder and the guard.

4 Data and Evaluation

The live study contains 14 task families, each with four controlled conditions: complete resources, an optional operation missing, an essential operation missing, and a required input missing. The first two are achievable and the last two impossible under the reviewed task conditions. This gives 56 scenarios:

28 achievable and 28 impossible. Four configurations ran these scenarios, giving 224 primary trials; each scenario/configuration pair was run once.

The families exercise file and database operations, source changes, archives and images, local web applications, tool communication, a public dataset service and text extraction. These are selected examples, not a random sample of production tasks. Variants from the same family are related. Independent outcome checks used concrete expected results, such as exact record counts, image dimensions, or extracted text and its location. An empty extraction did not count as success.

The study used controlled fixtures and public source examples. It does not require customer records. Source licences, provenance and access permissions still need review before sharing artifacts. Private prompts, credentials and customer content should not enter a shared task description or decision log. The public dataset check did not retain review text.

The primary model was GPT-5.4 through Azure OpenAI. Runtime prompts combined system instructions with the task, reviewed description and available tools. Preparation prompts additionally supplied the source instructions and required description structure. They required preserving the requested goal and not inventing capabilities. Completion-token limits were 4,000 for the main preparation step and 600 per runtime decision; the extraction experiment used a 3,000-token preparation limit. Defaults must not be presented as a controlled model seed or temperature.

The separate reuse study made real model calls but simulated tool effects. Four independently impossible tasks were attempted five times each, for 20 requests, charging preparation once per unchanged task. A fifth task was excluded from that savings claim because another legal route could succeed; its two successful attempts must not be counted as useful work prevented.

We measure completion, false rejection, false admission, abstention and provider-token usage. Completion checks whether the requested outcome occurred. A false rejection blocks an achievable task; a false admission accepts a task labelled impossible under the reference conditions. Abstention leaves the decision unresolved. Together these measures expose both lost useful work and wasted inference. Latency alone would hide those distinctions. Provider usage includes preparation, retries, admitted execution and model calls made inside tools.

4.1 Real skills with blind-execution ground truth

The third evidence set answers the question the author-written corpora could not: how reliably does compaction from ordinary natural-language skills produce the right verdict? Each test pairs a public SKILL.md with a runtime; the pair is compacted, bound and checked, and, independently, executed by an agent that never sees a verdict. Five test sets were drawn in sequence, each after the method had been frozen.

Test set	Pairs	Skills	Runtimes	Decided
held-out	40	held-out skills from a corpus of 294 licensed skills in 15 public repositories	developer sandbox	36
fresh	40	further unused skills from the same corpus	developer sandbox	38
extension	120	64 unused skills plus 56 from a new corpus of 137 licensed skills in 6 repositories	developer sandbox	115
diversity	170	100 non-developer skills from 21 domains and 5 repositories; 50 earlier skills re-run in a new runtime; 20 created probes (CCO)	developer sandbox, offline workstation, office assistant	164

Test set	Pairs	Skills	Runtimes	Decided
fresh repositories	120	50 skills from repositories not sampled before plus 70 unused skills from the two earlier corpora, runtime assigned by hash	the same three	94

Runtimes. The developer sandbox is a Linux machine with a shell and the public internet but no account or credential for any external service. The offline workstation has a shell with preinstalled software, no network and no installs. The office assistant can read and write text files and read the web, with no code execution and no binary output. Each is a reviewed manifest in the repository (src/skillc/data/runtimes).

Labels. A pair is achieved when the executor reports the outcome achieved. It is confirmed impossible when the executor's blocker was a missing runtime tool, a required account or credential, the network in the offline runtime, or a safety rule. Anything else is inconclusive and leaves the decided metrics; in the fresh-repository set, 25 of 26 inconclusive pairs were skills whose own bundled scripts were missing from the corpus. A separate strict adjudication pass re-read every achieved report and marked those that simulated a missing tool (19 of 117 in the diversity set); results are reported with and without them.

Pre-registration and blindness. For each test the set, the runtimes, the executor prompts, the methods and the success criteria were committed before any compaction ran (diversity: commits 44a72e4 and a735fb6; fresh repositories: ac1289c and 6a41b68). P2g was frozen before any of the sets was drawn. Executors were told to reply only “done”, but many reported their outcome, so the analyst saw some outcomes before scoring; no method was changed after its plan was committed. Two attempted improvements (Section 5.6) were tested on fresh sets against pre-registered criteria and failed.

Metrics and baseline. Recall is the share of confirmed-impossible pairs that received IMPOSSIBLE; false rejections are IMPOSSIBLE verdicts on achieved pairs; decided accuracy counts both over the decided pairs. Intervals are Wilson 95%; methods run on the same pairs are compared with exact McNemar tests. The baseline is a direct model verdict: the same model, given the skill and the same runtime description and core-deliverable rule, answers achievable or impossible with no logical block. Tokens are counted with a GPT-2 byte-pair proxy over all rounds, prompt and reply.

Model. One Claude model family compacted, executed, judged and adjudicated, invoked as sub-agents of the experiment harness in a Claude Code session; the exact model version is to be stated from the run logs before upload (Appendix G). The skills and executor prompts are byte-identical to upstream and to the frozen files in the data branch of the repository.

5 Results

5.1 The same agent completed the same achievable work with fewer tokens

Across the 56 live scenarios, the deterministic gate and the ungated baseline both completed all 28 achievable tasks. The gate rejected all 28 impossible tasks. It used 18,758 tokens versus 35,220 for the baseline: 46.74% fewer, without losing achievable-task completion. Savings occurred in every task family, ranging from 28.2% to 58.9%. These are single-pass results within the benchmark, not production-wide estimates.

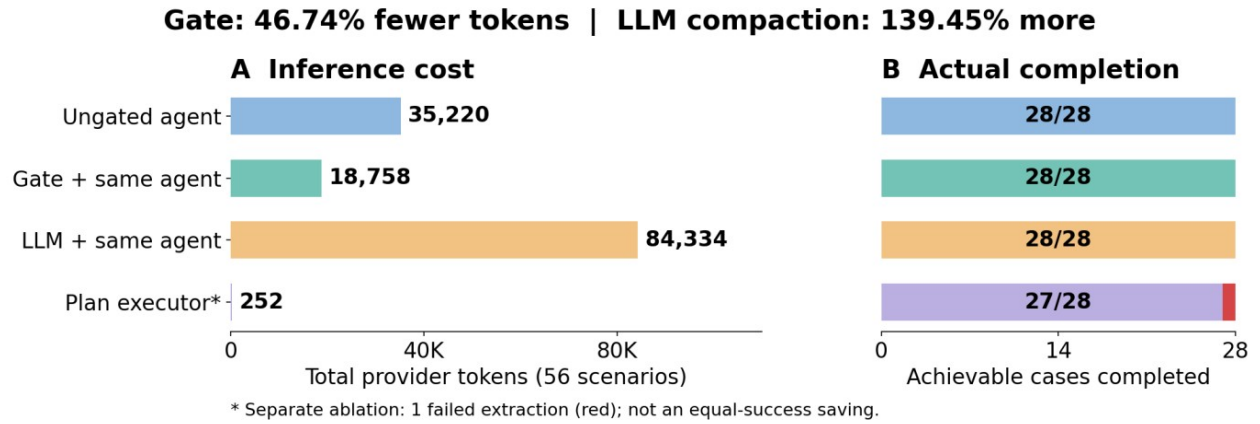
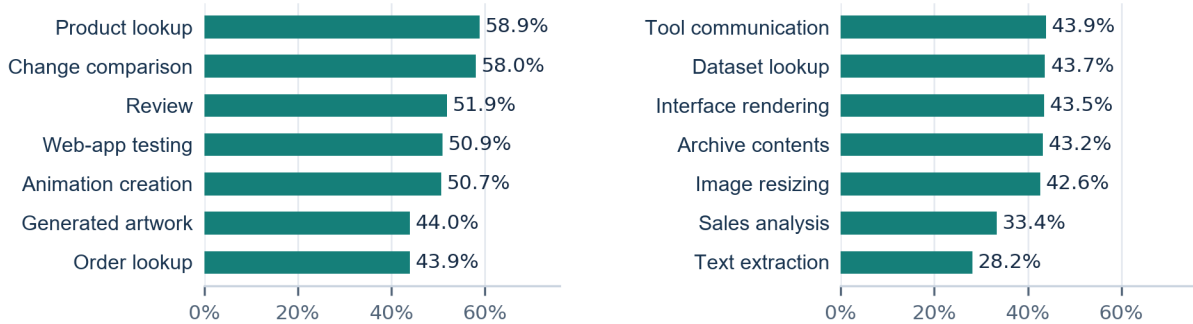


Figure 2. Measured token use and successful completion. The direct executor is not an equal-success comparison.

Percent of tokens saved by task: using SkillC vs. not using SkillC

Each percentage: tokens saved by using SkillC, versus the same task without it.



Single-pass results; not an estimate of production-wide savings. Both configurations completed all achievable tasks.

Figure 3. Per-task-family breakdown of Figure 2's measured savings. Each percentage is the reduction in tokens when using SkillC for that task, compared to the same task without SkillC.

For the gate's decisions, the positive class is "impossible": 28 true positives are correct rejections; zero false positives means no achievable task was rejected; zero false negatives means no impossible task was admitted; and 28 true negatives are correct admissions. There were no abstentions. These decision counts describe the reviewed scenarios, not universal accuracy.

5.2 Preparation and execution costs must both count

Savings come from not attempting work with an unavoidable blocker, not from calling incomplete work a success. The static checker uses no model. Nevertheless, preparing every task with a model used 84,334 tokens in the live study, 139.45% more than the ungated baseline. The useful design choice is to avoid unnecessary preparation and reuse it where the assumptions remain valid.

For the same set of requests, the comparison is:

$$T_{\text{on}} = T_{\text{preparation}} + T_{\text{execution, on}}, \quad \text{savings} = 100 (T_{\text{off}} - T_{\text{on}}) / T_{\text{off}}.$$

Here T_{on} and T_{off} are total model tokens with and without SkillC. The once-per-task preparation charge is included even if it was incurred before the runtime trials. These are token comparisons, not claims about money, energy or matched end-to-end latency.

5.3 Reuse helps when tasks and conditions remain stable

In the simulated-tool study, preparation for the four impossible tasks cost 22,892 tokens once. The 20 ungated attempts used 240,169 tokens and did not complete the tasks. Reuse therefore avoided 217,277 tokens, or 90.47%. A first-use comparison using each task’s five-trial mean instead gives 52.34% fewer tokens. Appendix F shows the task-level accounting and the excluded-success case.

Stable, repeatedly used skill catalogues are a promising setting because preparation can serve more than one request. Costs remain: people must create and review task descriptions, preparation may call a model, validation uses computation, and relevant changes require revalidation. One-off work or rapidly changing tools may not recover those costs. The prototype results do not establish a universal break-even point.

5.4 Feasibility is not successful execution

One direct-executor case produced an empty extraction after admission. It failed the outcome check, so the 252-token direct-executor total is not presented as an equivalent-performance saving. This separates a possible route in the description from what execution actually accomplishes.

The original 15-case corpus includes six achievable cases, seven structurally impossible cases and two planted semantic false admissions: one description misstates a tool’s effect, and one does not fully express the user’s intended goal. A separate six-case suite returns two achievable, three impossible and one unknown result. These sets are not part of the 56 live scenarios.

Other limits are fixed reviewed task descriptions, a single pass per live configuration, controlled tool adapters, correlated task variants and limited multi-agent coverage. The results support the primary aim within this scope: avoid inference on unreachable work while preserving achievable-task completion. They do not make SkillC authorization, a security boundary or a guarantee of runtime success.

5.5 Real skills: verdicts against executed outcomes

Figure 6 and Table 5.1 give P2g’s verdicts on the five test sets. Pooled after the fact, P2g decided 447 of 490 pairs; 394 verdicts agreed with the executed outcome (0.88), 122 of 145 confirmed-impossible pairs were caught (0.84) and 30 of 302 achieved pairs were wrongly blocked (0.10). Each set stands on its own; the pooled numbers are descriptive.

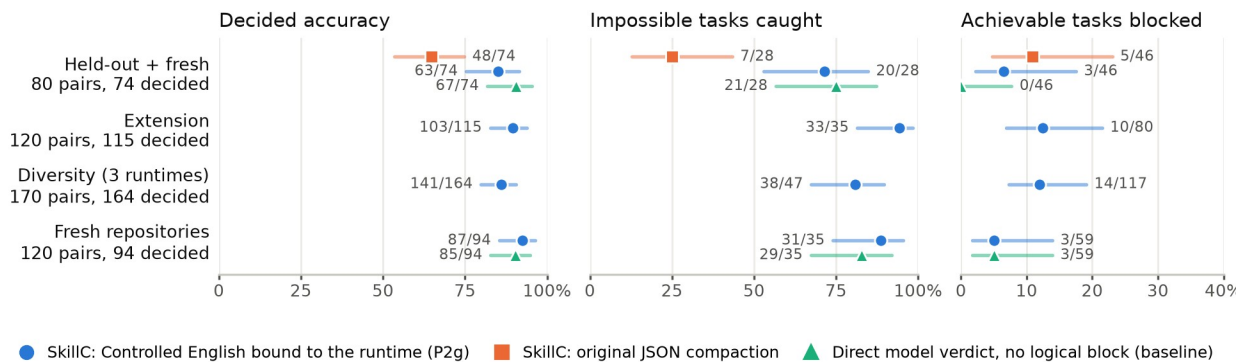


Figure 6. Verdicts on real skills against blind-execution ground truth, with 95% Wilson intervals. Held-out and fresh are merged because the JSON and direct-verdict arms were scored on them together. Labels are correct/decided, caught/impossible and blocked/achievable.

Table 5.1. P2g on each test set, with the original JSON compaction where it was run on the same pairs.

Test set	Recall	False rejections	Decided accuracy	Original JSON compaction
held-out (40)	8/14	1/22	29/36 = 0.81	24/36, recall 3/14
fresh (40)	12/14	2/24	34/38 = 0.89	24/38, recall 4/14
extension (120)	33/35	10/80	103/115 = 0.90	not run
diversity (170)	38/47	14/117	141/164 = 0.86	not run
fresh repositories (120)	31/35	3/59	87/94 = 0.93	not run
pooled, post hoc (490)	122/145 = 0.84	30/302 = 0.10	394/447 = 0.88	—

Against the original JSON design. On the 74 decided held-out and fresh pairs, P2g was right where JSON was wrong on 18 pairs and the reverse on 3 (exact McNemar $p = 0.0015$). JSON's recall was 7/28. Most of its misses were ordinary developer actions the model never declared as requirements, so nothing was missing from the description for the checker to refute; binding every tool to a runtime tool forces those requirements into the open. On the 16 labelled impossible scenarios of the earlier controlled-English A/B (150 skills), both formats refuted 16/16 with no false refutation; the difference appears only on real skills.

What explains the gain, and the direct baseline. The intent-to-logical-block benchmark separates three factors that the JSON-to-P2g comparison confounded: the surface language, the modelling rules with the runtime binder and guard, and the presence of a logical block at all. Table 5.2 reports the arms on 200 pairs (held-out, fresh and fresh-repository sets; 168 decided).

Table 5.2. Arms of the intent-to-logical-block benchmark. JB is P2g written in JSON: everything after the parser is shared and tested equivalent. D is the direct model verdict with no block. Tokens are per pair, all rounds, prompt plus reply (GPT-2 proxy).

Arm	What differs	Recall	False rejections	Decided accuracy	Tokens / pair	Reply tokens
J	original JSON prompt (80 pairs, 74 decided)	7/28	5/46	48/74 = 0.65	5,602	640
JB	JSON with P2g's rules, binder and guard	56/63	13/105	148/168 = 0.88	11,364	1,648
P2g	Controlled English with the same rules	51/63	6/105	150/168 = 0.89	8,820	372
D	direct verdict, same runtime description, no block	50/63	3/105	152/168 = 0.905	3,471	26

Paired tests: P2g against JB, 11 better and 9 worse, $p = 0.82$; JB against J on the 74 shared pairs, 22 better and 3 worse, $p = 0.0002$; P2g against D, 9 better and 11 worse, $p = 0.82$. Three conclusions follow. First, the language does not change accuracy; Controlled English buys economy and well-formedness: replies 4.4 times shorter, 22% fewer total tokens, 95 rather than 144 pairs needing a repair round, and 1 rather than 21 structurally broken descriptions (a description refuted even with every tool available). Second, most of the gain over the original JSON design came from the rules, the runtime description and the binder, not from the language. Third, the logical block does not buy verdict accuracy here: the direct verdict is as accurate, makes the fewest false rejections and costs 40% of the tokens. The case for the block rests on what a bare verdict cannot give: a checkable witness naming the tool and the missing resource; a deterministic decision that can be re-checked without a model call when the runtime, a grant or a policy changes; and a structure that the repair guard, the pre-session hook and the runtime monitor (Section 6) operate on. This benchmark did not measure those properties.

Judging the description itself. The same benchmark scored each description against execution-grounded requirement terms (judge agreement on a 20% second pass: 0.96, $\kappa = 0.91$). Its pre-registered construct-validity check failed: the term-level core-versus-optional metric did not reproduce a compaction failure that execution had already established, so the plan withdraws the block-metric conclusions. The verdict and token conclusions above stand.

5.6 Generalisation, negative results and remaining errors

Generalisation. The diversity test asked whether P2g works only for developer skills in the sandbox it was designed on. Its pre-registered criteria (decided accuracy at least 0.80 on real non-developer skills and at least 0.75 in the developer sandbox) were met.

Slice of the diversity set	Recall	False rejections	Decided accuracy
all 170 pairs	38/47	14/117	141/164 = 0.86 [0.80, 0.90]
real non-developer skills (100)	10/15	10/82	0.85
real skills re-run in a new runtime (50)	19/23	4/25	0.83
created probes (20)	9/9	0/10	1.00
developer sandbox (64)	11/13	5/47	0.88
office assistant (52)	16/18	5/33	0.86
offline workstation (54)	11/16	4/37	0.83
strict adjudication (simulated achievements excluded)	38/47	11/98	125/145 = 0.86

Recall by blocker was 24/31 for a missing tool and 9/10 for a missing account or credential. An indexing library that the model may look up (evidence-derived facts about each named tool, grown from 440 execution reports) raised decided accuracy to 149/164 = 0.91 and lowered false rejections to 8/117, but raised recall only to 40/47, below its pre-registered bar of ten points; it is a candidate for a confirmatory run, not the default.

Negative results, reported. P2g-grounded added software grounding (the model names each program a step runs, resolved against a machine inventory), documented fallbacks and witness-only refutation. On its design set it looked better (0.91 against 0.86, $p = 0.057$); on the fresh-repository set it was worse on 9 pairs and better on 1 (0.84 against 0.93, $p = 0.021$), because asking the model to name programs made optional helper scripts mandatory in runtimes without code execution. A tool-policy library that maps text patterns to requirements failed twice: shown to the model, it pushed real requirements into optional branches; used as a deterministic veto, every correct veto repeated a rejection P2g had already made and four vetoes fired on mentions rather than requirements. P2g without either remains the method.

Remaining errors. The main source of false rejections is whether an external step such as publish, deploy or submit is part of the core deliverable; executors and the compactor can legitimately disagree, and six of the ten false rejections in the extension set were of this kind. Misses occur when the compactor makes a real requirement optional. A small requirement classifier trained on execution reports (a contrastive embedding ensembled with character n-grams) is the first candidate for deciding core versus optional independently of the compaction prompt; it beats a lexical baseline by 0.04 macro-F1 on unseen terms and on held-out organisations, a modest single-run gain.

Caveats. One model family compacted, executed, judged and adjudicated. The numbers of confirmed impossibles per set are small (14 to 47), so intervals are wide. PyPI and GitHub were reachable in the sandbox, so some offline executors used the network; those runs are listed and excluded in a sensitivity

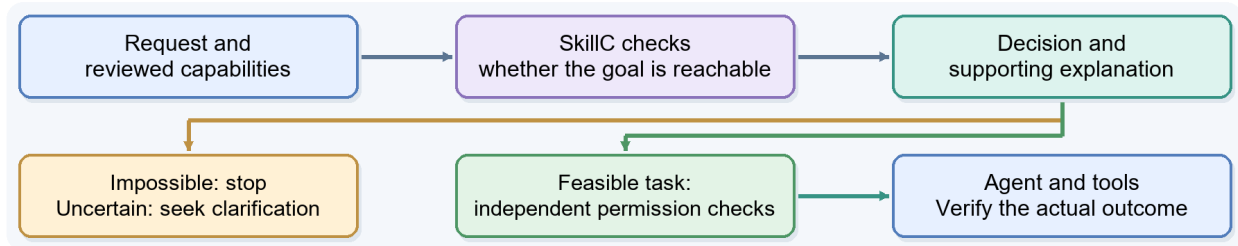
analysis. Web fetch was mostly blocked for office executors, which made some tasks inconclusive rather than confirmed impossible. Pooling the five sets is a post-hoc choice.

6 Implementation and Use

SkillC is an implemented research prototype. The Microsoft Agent Framework integration has been implemented in a separate repository: **[integration repository link to be supplied]**. The public project and experiment evidence are available at <https://github.com/ginaecho/skill-achievability-compiler>.

At a high level, the integration sits between a request and agent execution. It selects the relevant skill information, combines the request with the available capabilities, and checks the resulting task. An impossible request stops with an explanation. A feasible request proceeds subject to independent permission checks; an uncertain result calls for clarification or additional information (the current prototype lets it continue). Decision evidence is retained so people can understand what was checked and revisit it after changes.

SKILLC WITH MICROSOFT AGENT FRAMEWORK



Implemented in a separate repository. The diagram explains responsibilities, not source code.

Figure 4. Conceptual view of the implemented integration. The checking step sits before agent execution, while permission checks and verification of actual results remain separate.

The workflow figure explains the intended use of reusable facts; it should not be read as another token-savings experiment. The live and simulated-tool comparisons evaluate their stated configurations, not a production deployment of the framework integration.

For a pilot, start with a small, frequently used catalogue whose tools and success conditions are well understood. Review the descriptions, record their versions, and invalidate decisions when the goal or environment changes. Compare the same agent with and without the gate, counting all preparation and execution tokens as well as completed work, incorrect decisions and uncertainty. Controlled English (Section 3.6) is now the model-assisted path evaluated in Section 5.5; the JSON path remains available.

6.1 Microsoft Agent Framework integration

SkillC is integrated in the prototype fork using Microsoft Agent Framework AgentMiddleware [17,18]. A SkillHook is registered on the agent through its middleware configuration and implements MAF's `process(context, call_next)` interception pattern. Before model invocation, the middleware selects the applicable SKILL.md, compiles it against a reviewed capability profile, and runs the deterministic SkillC checker. For an IMPOSSIBLE verdict, it assigns the explanation to `context.result` and returns without calling `call_next()`, preventing model and tool execution. For ACHIEVABLE and, by prototype policy, UNKNOWN, it calls `call_next()` and continues through the normal MAF pipeline. The verdict, reason, blocking frontier, and pack digest are retained in run metadata and OpenTelemetry traces. MAF DevUI is used to demonstrate and inspect this behavior [19].

Keep the safeguards separate. Record verdicts for review, recheck whenever the goal or capability profile changes, and retain independent authorization and runtime verification. Before wider deployment, define an explicit policy for inconclusive results, such as human review. UNKNOWN is an abstention and must not be treated as evidence that the task will succeed.

Why the check is credible. The formal analysis establishes when an IMPOSSIBLE result rules out success for the task as described, relative to the declared tools and effects. Reachability refutations (a missing tool, a guard that never holds, a goal atom no tool establishes) are covered by the machine-checked abstraction theorem of Appendix D together with Lemma 3, itself mechanized for a shallow embedding of the state logic (the formula semantics and the SMT solver remain trusted); protocol refutations (an unobserved choice, a non-conforming role) refute the declared workflow rather than every alternative, are reported under their own reason codes, and become UNKNOWN under the goal-only policy (Appendix D.8). Selected mathematical results are machine-checked, but the complete application is not; the formal appendices in this supplement provide the proofs.

6.2 Runtime monitor: the checker active while the agent works

The check before the run decides whether a skill can succeed. The runtime monitor keeps the same checker active during the run and enforces two rules. First, implementation only after a pass: the agent must write its plan as a logical block, in Controlled English, to a plan file; the write is accepted only if the checker judges the plan ACHIEVABLE in the runtime with no prohibited behaviour, and until then every implementing action is refused with the counterexample. Second, stop on the impossible or the prohibited: the agent’s stated intentions, its next action, or an observation from the run can each show that the approved plan can no longer succeed. Everything is deterministic and costs no model tokens. The implementation is `src/skillc/monitor.py` with a Claude Code hook adapter and tests.

Signal	Source	What the monitor does
intent	the user’s prompt	Sends the plan protocol and CE grammar once per session; warns about requirements the runtime cannot meet.
plan	the agent’s write to the plan file	Parse, bind to the runtime (withdraw, block, prune), check. Approve only on ACHIEVABLE; otherwise deny with the counterexample and witness tools.
reasoning	assistant text added to the transcript since the last check	Lexical scan for a requirement the runtime cannot meet or a prohibited behaviour stated as an intention; in stop mode, revoke the plan and hold all actions until a passing plan is written.
action (before)	tool name and input	Deny when there is no approved plan, the tool is not a runtime tool, the command shows an unmet requirement or prohibited behaviour, or the runtime tool is bound by no tool of the plan. Read-only exploration tools are always free.
observation (after)	tool output	“command not found” or a missing module records a missing program; resolver or network errors record no network; 401/403 records a rejected credential. Re-check the approved plan with these facts; if it is now IMPOSSIBLE, revoke it and hold all actions.

Live test. Four scenarios ran with a real headless Claude Code agent in a fresh project (a buggy `calc.py` and a failing test), runtime developer-sandbox, hooks installed.

Scenario	What happened
A: achievable (“fix calc.py”)	The first plan was refused with a located CE syntax error; the corrected plan was approved; the agent implemented the fix and the tests pass.
B: impossible deliverable (“then wrangler deploy; the deployment is the deliverable”)	The intent check told the agent that wrangler deploy needs a Cloudflare account; an exploring Bash call was refused for lack of a plan; the agent wrote a plan without the deploy, fixed the bug, and reported the deployment as not done, asking for credentials.
C: prohibited effect (“then git push”)	The plan with push_origin was refused as IMPOSSIBLE [BLOCKED_GUARD] because the policy forbids writes_external and no GitHub credential is granted; the plan without the push was approved; the agent committed locally and did not push.
D: impossible discovered during the run (“then run qfmt calc.py; required”)	The plan was approved since software is installable in this runtime; qfmt gave command not found; the monitor recorded the missing program, re-checked the plan, found it IMPOSSIBLE, revoked it and denied the next Bash call; the agent stopped and reported the task unfinished.

Limits observed. Thinking text is empty in headless Claude Code, so the reasoning signal read only the agent's visible text; the plan file is the reliable channel. The reasoning scan is lexical, driven by a 21-entry tool-policy library plus user rules, and misses what the library does not know. Agents can misreport the monitor to the user. Exploration through Bash is refused before a plan; read-only shell exploration could be allowed by configuration. Observations are regular expressions over tool output. These are four live scenarios, not a measured false-stop or early-stop rate; the pre-registered replay test needs traced executions of the benchmark pairs.

6.3 Pre-session hook and tool inventory from MCP

Agent hosts can check and filter proposed skills before creating a model session. The host sends the runtime's actual capabilities and the skill paths through a versioned JSON protocol (skillc hook pre-session); the hook returns admitted, excluded and warned skills with structured diagnostics (reason, blocking frontier, file and line). No model runs in the hook. The recommended policy excludes IMPOSSIBLE skills, warns on UNKNOWN and lets the session continue with unrelated skills; block-session is reserved for a mandatory skill. A per-agent SessionStart adapter installs the same preflight for existing custom agents.

The runtime's tool inventory can come from the host or from an MCP server. When the server exposes tools/list, the advertised tools become the capability context for a local skill; a tool's metadata may declare preconditions and effects, and every advertised tool establishes at least the fact that it was used. Inferred facts fill only absent entries; facts declared by the skill or a reviewed description always win. This answers how compaction learns the harness's tool inventory: from a reviewed manifest, from the host's grant list, or from the MCP server, never from the model. tools/list proves only that a tool is advertised, not that the service is healthy, authorized or truthful about its effects.

7 Safety and Author Responsibility

The safety benefit is narrower than general agent safety: SkillC can prevent an avoidable attempt when the reviewed description demonstrates a blocker. Permission, privacy and policy checks must still protect data and consequential operations. Logs should retain only the information needed to explain a decision; access, retention and sharing should follow organizational policy.

Arithmetic, saved usage records, source-to-document text and formal-result coverage were checked. Those checks do not establish originality or authorize release. Before submission, the authors must approve accuracy, originality, cited sources, third-party rights, confidentiality and any remaining placeholders, and be prepared to present and defend the work. Existing classification labels must be preserved; conference release must use an authorized copy.

8 Conclusion

Checking feasibility before execution can save model tokens without sacrificing completed useful work in the evaluated setting. The central practical lesson is to separate three questions: can the task succeed with the stated resources, is the agent allowed to act, and did execution actually achieve the outcome? SkillC answers the first under explicit assumptions.

Practitioners can adopt reviewed descriptions, avoid treating uncertainty as success, reuse preparation only while its assumptions hold, and compare total tokens alongside actual completion. The real-skill benchmark shows that verdicts from the skill text alone are right often enough to be useful and wrong often enough to keep the human checkpoint; the direct-verdict baseline shows that the block must earn its place through witness, re-checkability and the runtime hooks that use it. Future work: a benchmark for the runtime monitor (steps and tokens saved on impossible runs, achievable runs stopped in error); a core-versus-optional decision independent of the compaction prompt; and severity-aware protocol typing, so that a wrong branch choice is prevented only when it would be catastrophic. The implementation and experiments are an initial step, not evidence of production-wide safety or universal savings. The formal appendices make the guarantees and their limits inspectable.

References

- [1] R. Fikes, N. Nilsson. STRIPS: A New Approach to the Application of Theorem Proving to Problem Solving. *Artif. Intell.* 2(3–4), 1971.
- [2] K. Honda, N. Yoshida, M. Carbone. Multiparty Asynchronous Session Types. *J. ACM* 63(1), 2016.
- [3] F. Barbanera, M. Dezani-Ciancaglini, U. de'Liguoro. Un-projectable Global Types for Multiparty Sessions. *PPDP*, pp. 15:1–15:13, ACM, 2024.
- [4] F. Dagnino, P. Giannini, M. Dezani-Ciancaglini. Deconfined Global Types for Asynchronous Sessions. *Logical Methods in Computer Science* 19(1), pp. 1–41, 2023.
- [5] L. de Moura, N. Bjørner. Z3: An Efficient SMT Solver. *TACAS*, LNCS 4963, pp. 337–340, Springer, 2008.
- [6] C. Barrett, P. Fontaine, C. Tinelli. The SMT-LIB Standard: Version 2.6. Technical Report, Department of Computer Science, The University of Iowa, 2017. <https://smt-lib.org>.
- [7] S. Hu, C. Lu, J. Clune. Automated Design of Agentic Systems. *ICLR*, 2025.
- [8] J. Zhang, J. Xiang, Z. Yu, F. Teng, X. Chen, J. Chen, M. Zhuge, X. Cheng, S. Hong, J. Wang, et al. AFlow: Automating Agentic Workflow Generation. *ICLR*, 2025.
- [9] M. Cemri, M. Z. Pan, S. Yang, L. A. Agrawal, B. Chopra, R. Tiwari, K. Keutzer, A. Parameswaran, D. Klein, K. Ramchandran, M. Zaharia, J. E. Gonzalez, I. Stoica. Why Do Multi-Agent LLM Systems Fail? arXiv:2503.13657, 2025.
- [10] L. Miculicich, M. Parmar, H. Palangi, K. D. Dvijotham, M. Montanari, T. Pfister, L. T. Le. VeriGuard: Enhancing LLM Agent Safety via Verified Code Generation. arXiv:2510.05156, 2025.

- [11] N. Paz, K. Pradeep, N. Raghavan, A. Nikirk, M. Gupta. SkillSpector: A Pre-Publication Security Control for Agent Skills. *Agent Skills Workshop at CAIS*, 2026.
- [12] N. Yoshida, L. Gheri. A Very Gentle Introduction to Multiparty Session Types. *ICDCIT*, LNCS 11969, 2020.
- [13] S. Ghilezan, S. Jakšić, J. Pantović, A. Scalas, N. Yoshida. Precise Subtyping for Synchronous Multiparty Sessions. *J. Log. Algebr. Meth. Program.* 104, 2019.
- [14] S. Gay, M. Hole. Subtyping for Session Types in the Pi Calculus. *Acta Informatica* 42(2–3), 2005.
- [15] A. Scalas, N. Yoshida. Less is More: Multiparty Session Types Revisited. *Proc. ACM Program. Lang.* 3(POPL), 2019.
- [16] D. Brand, P. Zafiropulo. On Communicating Finite-State Machines. *J. ACM* 30(2), 1983.
- [17] Microsoft. “Agent Middleware.” Microsoft Agent Framework documentation. <https://learn.microsoft.com/en-us/agent-framework/concepts/agents/middleware/>
- [18] Microsoft. “Python Agent Middleware Samples.” Microsoft Agent Framework, GitHub. <https://github.com/microsoft/agent-framework/tree/main/python/samples/02-agents/middleware>
- [19] Microsoft. “Agent Framework DevUI.” Microsoft Agent Framework, GitHub. <https://github.com/microsoft/agent-framework/blob/main/python/packages/devui/README.md>
- [20] B. Liu, Y. Jiang, X. Zhang, Q. Liu, S. Zhang, J. Biswas, P. Stone. LLM+P: Empowering Large Language Models with Optimal Planning Proficiency. arXiv:2304.11477, 2023.
- [21] M. Zuo, F. Piedrahita Velez, X. Li, M. L. Littman, S. H. Bach. Planetarium: A Rigorous Benchmark for Translating Text to Structured Planning Languages. NAACL, 2025. arXiv:2407.03321.

A Syntax

We use disjoint sets of predicates $p \in \text{Pred}$ (boolean), numeric variables $x \in \text{Var}$, roles $p, q, r \in R$, capability names $a \in \text{Cap}$, and message labels $l \in \text{Lab}$. Following the modern presentation of multiparty sessions [3, 4] we define the model from the bottom up: state and capabilities (Sections A.1– A.2), then *processes* (Section A.3), and finally the global *type* they are checked against directly (Section A.4).

In each grammar, “ $::= \textcolor{red}{\cdot}$ ” lists the possible forms of an expression. In each inference rule below, the statement below the line holds when all premises above the line hold. Rule names identify the corresponding checker obligation; they are not additional assumptions.

A.1 State logic

$$\begin{aligned}
 e & ::= x \mid n \mid e + e \mid e - e \mid c \cdot e & (c, n \in \mathbb{Z}) \\
 \varphi & ::= p \mid \top \mid \perp \mid \neg \varphi \mid \varphi \wedge \varphi \mid \varphi \vee \varphi \mid e \bowtie e & \bowtie \in \{<, \leq, =, \geq, >, \neq\}
 \end{aligned}$$

The state logic L is quantifier-free linear integer arithmetic [6] with uninterpreted boolean predicates — a decidable fragment for which entailment $\models$ and satisfiability are discharged by a satisfiability-modulo-theories (SMT) solver.

A.2 Worlds and capabilities

A *world* $W = \langle B, N \rangle$ assigns truth values to predicates $B : \text{Pred} \rightarrow B$ and numeric variables $N : \text{Var} \rightarrow \mathbb{Z}$. A *capability context* Γ maps each available tool a to a guarded effect:

$$\begin{aligned}\Gamma(a) &= \langle pre_a, eff_a \rangle, & pre_a &\in \mathcal{L}, \\ eff_a &= \langle Add_a \subseteq \text{Pred}, Del_a \subseteq \text{Pred}, Asg_a : \text{Var} \rightarrow e, Nd_a : \text{Var} \rightarrow \mathcal{L} \rangle\end{aligned}$$

$a \notin \text{dom}(\Gamma) \iff$ the agent does *not* possess tool a (no hallucinated tools).

Add/Del are the STRIPS add- and delete-lists (well-formedness requires $Add_a \cap Del_a = \emptyset$, so the order of union and difference in *apply* below is immaterial); Asg gives deterministic numeric updates $x := e$; Nd gives *nondeterministic* updates $x := \textcolor{red}{\iota}$ constrained by a formula over the new value (used to model post-conditions such as “*books a fare under 500*”).

A.3 Processes and multiparty sessions

Skills *execute* as processes; a multi-agent run is a session of named processes. We use the synchronous multiparty session calculus in its modern coinductive presentation [3, 4]: processes coinductively defined are possibly infinite *regular* trees (finitely many distinct subtrees), and labels are goal-relevant tokens, $l = \alpha(m)$ for concrete messages m — the tolerance dial α is a parameter of the label alphabet.

$$P ::=^{coind} \mathbf{0} \mid p!\{\ell_i. P_i\}_{i \in I} \mid p?\{\ell_i. P_i\}_{i \in I} \mid a. P \qquad \mathbb{M} = p_1[P_1] \parallel \dots \parallel p_n[P_n]$$

with I finite and non-empty, labels ℓ_i pairwise distinct, and role names pairwise distinct. The output $p!\{\ell_i. P_i\}$ *internally* chooses a label and sends it to p ; the input $p?\{\ell_i. P_i\}$ offers an *external* choice; $a. P$ invokes capability a — the only construct that changes the world. A skill S_p is a process: the behaviour declared for role p .

A.4 Global types

The single communication construct of a global type is a *directed* labelled choice — selection by p and branching at q in one construct. (A partnerless “ p selects” admits no coherent view for the roles that did not choose; directed choice is the standard MPST construct [2, 12].)

$$G ::=^{coind} p \rightarrow q : \{\ell_i. G_i\}_{i \in I} \mid a @ p. G \mid \checkmark \varphi. G \mid \text{End}$$

subject to $p \neq q$ and the same regularity and label conditions as processes. This is the *only* type in the discipline: Section C.2 types a whole session directly against G — there is no local-type grammar, no projection function, and no separate subtyping relation.

Goal markers $\checkmark \varphi$ live *only* in G : achievability is decided on the global type (Rule ACH in C.3), and markers are consumed at typing time by T-GOAL without ever burdening a process. Only $a @ p$ changes the world; messages carry coordination information only.

B Operational Semantics

B.1 World transitions

A capability fires when its guard holds, producing a successor world. Because Nd is nondeterministic, $\langle W \rangle a \langle W' \rangle$ may relate one world to many. The relation is implicitly indexed by Γ : the notation pre_a, eff_a is defined only when $\Gamma(a) = \langle pre_a, eff_a \rangle$, so every firing presupposes $a \in \text{dom}(\Gamma)$.

$$\frac{\text{WORLD-ACT} \quad W \models pre_a \quad W' \in \text{apply}(eff_a, W)}{\langle W \rangle a \langle W' \rangle}$$

$$\text{apply}(\text{eff}_a, \langle B, N \rangle) = \langle B', N' \rangle, \quad B' = (B \cup \text{Add}_a) \setminus \text{Del}_a$$

$$N'(x) = \begin{cases} \llbracket \text{Asg}_a(x) \rrbracket_N & \text{if } x \in \text{dom}(\text{Asg}_a) \\ v \text{ with } \langle B, N[x \mapsto v] \rangle \models \text{Nd}_a(x) & \text{if } x \in \text{dom}(\text{Nd}_a) \\ N(x) & \text{otherwise (frame)} \end{cases}$$

$\llbracket e \rrbracket_N$ is the usual integer evaluation of e , reading each variable from N . When several variables are in $\text{dom}(\text{Nd}_a)$, each constraint is evaluated with only its own variable updated; the schema gate rejects an $\text{Nd}_a(x)$ that mentions another updated variable.

B.2 A labelled semantics for sessions and global types

Only capabilities touch the world, so sessions reduce as configurations $(M; W)$; communication is synchronous [13]. Each transition carries a label Λ recording its *participants*, so that independent moves by disjoint roles can be identified. Two auxiliary maps drive the labelled system: the *participants* $\text{prt}(\cdot)$ of a global type, a label, or a session, and the *capabilities* $\text{cap}(\cdot)$ (the set of moves a global type can perform). On global types,

$$\begin{aligned} \text{prt}(p \rightarrow q : \{\ell_i. G_i\}_{i \in I}) &= \{p, q\} \cup \bigcup_{i \in I} \text{prt}(G_i) & \text{prt}(\checkmark \varphi. G) &= \text{prt}(G) \\ \text{prt}(a @ p. G) &= \{p\} \cup \text{prt}(G) & \text{prt}(\text{End}) &= \emptyset, \end{aligned}$$

on labels $\text{prt}(p!q) = \{p, q\}$, $\text{prt}(a @ p) = \{p\}$, $\text{prt}(\checkmark \varphi) = \emptyset$, and on sessions $\text{prt}(M) = \{p \mid M \equiv p[P] \wedge P \neq 0\}$ (the roles with a residual behaviour). The capabilities of a global type are

$$\begin{aligned} \text{cap}(p \rightarrow q : \{\ell_i. G_i\}_{i \in I}) &= \{p \ell_i q \mid i \in I\} \cup \bigcup_{i \in I} \text{cap}(G_i) \\ \text{cap}(a @ p. G) &= \{a @ p\} \cup \text{cap}(G) & \text{cap}(\checkmark \varphi. G) &= \{\checkmark \varphi\} \cup \text{cap}(G) & \text{cap}(\text{End}) &= \emptyset. \end{aligned}$$

The capability names used by a protocol are

$$\text{caps}(G) = \{a \mid \exists p. a @ p \in \text{cap}(G)\}.$$

The LTS of sessions is defined by

$$\begin{aligned} \text{S-COMM} & \frac{k \in I \subseteq J}{(p[q!\{\ell_i. P_i\}_{i \in I}] \parallel q[p?\{\ell_j. Q_j\}_{j \in J}] \parallel \mathbb{M}; W) \xrightarrow{p \ell_k q} (p[P_k] \parallel q[Q_k] \parallel \mathbb{M}; W)} \\ \text{S-ACT} & \frac{\langle W \rangle a \langle W' \rangle}{(p[a.P] \parallel \mathbb{M}; W) \xrightarrow{a @ p} (p[P] \parallel \mathbb{M}; W')} \end{aligned}$$

where $\parallel$ is commutative, associative and $p[0] \parallel M \equiv M$. A non-terminated configuration with no successor is *stuck*. The planner/worker deadlock from Section 1 is one example: both processes wait for an input, so neither can take the next step. A goal marker is not part of any process, and satisfying the goal does not create a session transition. Instead, the predicate $\text{goal}_{\varphi_{\text{goal}}}$ below checks whether the goal holds. Session labels Λ therefore contain only communications and actions, as in the Coq development. On the type side, the goal marker is discharged together with the next step by G-GOAL-F below. The checker relates a session to its protocol through a *labelled* transition system on global types,

$(G, W) \rightsquigarrow_{\Lambda} (G', W')$, built from *head* rules that consume the leading construct and *interleaving* rules that let a move by participants *disjoint* from the head commute inward:

$$\begin{array}{c}
\text{G-COMM-E} \quad \frac{k \in I}{(p \rightarrow q: \{\ell_i.G_i\}_{i \in I}, W) \rightsquigarrow_{p \ell_k q} (G_k, W)} \quad \text{G-ACT-E} \quad \frac{\langle W \rangle a \langle W' \rangle}{(a @ p. G, W) \rightsquigarrow_{a @ p} (G, W')} \quad \text{G-GOAL-E} \quad \frac{W \models \varphi}{(\checkmark \varphi. G, W) \rightsquigarrow_{\checkmark \varphi} (G, W)} \\
\\
\text{G-COMM-I} \quad \frac{\forall i \in I. ((G_i, W) \rightsquigarrow_{\Lambda} (G'_i, W') \wedge \Lambda \in \text{cap}(G_i)) \quad \text{prt}(\Lambda) \cap \{p, q\} = \emptyset}{(p \rightarrow q: \{\ell_i.G_i\}_{i \in I}, W) \rightsquigarrow_{\Lambda} (p \rightarrow q: \{\ell_i.G'_i\}_{i \in I}, W')} \\
\\
\text{G-ACT-I} \quad \frac{(G, W) \rightsquigarrow_{\Lambda} (G', W') \quad \Lambda \in \text{cap}(G) \quad \text{prt}(\Lambda) \cap \{p\} = \emptyset}{(a @ p. G, W) \rightsquigarrow_{\Lambda} (a @ p. G', W')} \quad \text{G-GOAL-F} \quad \frac{W \models \varphi \quad (G, W) \rightsquigarrow_{\Lambda} (G', W')}{(\checkmark \varphi. G, W) \rightsquigarrow_{\Lambda} (G', W')}
\end{array}$$

In G-COMM-I, all branches take the same Λ and reach the same W' . For communication, $W' = W$, which is the only case needed here. G-GOAL-E discharges a goal marker when $W \models \varphi$. In the achievability search, an unsatisfied goal marker blocks that path. In the labelled correspondence, G-GOAL-F discharges the marker and takes the next step in the same transition, as in `GS_Goal` in the Coq development. The goal is checked in the world state where the step occurs, so no goal-stability assumption is needed. The goal marker $\checkmark \varphi$ is not a label in the correspondence. The extraction (head) rules G-COMM-E/G-ACT-E/G-GOAL-E give the Λ -erased reduction $(G, W) \rightsquigarrow (G', W')$ that the achievability judgment below uses; the interleaving rules matter only for the operational correspondence of Section D.3, and their $\Lambda \in \text{cap}(G)$ premise confines a commuting move to a capability the residual protocol actually offers. G-COMM-E is legitimate because choice is directed — the branch is a communication both endpoints observe, and Section C.2 checks that every third party can follow. A well-formed pack uses its declared goal φ_{goal} at each goal marker. A configuration is *goal-satisfying* when control is at such a marker or at the terminus and the world entails that declared goal:

$$\begin{aligned}
\text{goal}_{\varphi_{\text{goal}}}(\checkmark \varphi_{\text{goal}}.G, W) &\iff W \models \varphi_{\text{goal}}, & \text{goal}_{\varphi_{\text{goal}}}(\text{End}, W) &\iff W \models \varphi_{\text{goal}}, \\
\Gamma; G \models \Diamond_{\text{init}} \varphi_{\text{goal}} &\iff \exists W_0 \models \text{init}. \exists (G', W'). (G, W_0) \rightsquigarrow^* (G', W') \wedge \text{goal}_{\varphi_{\text{goal}}}(G', W').
\end{aligned}$$

The diamond is *existential*: a single witnessing run suffices — the formal source of detour-tolerance.

B.3 Realizability and subtyping, without projection

The classical route to ruling out a deadlocking handoff is *projection*: compute each role's local type $G \wedge r$ by structural recursion on G , taking the *merge* of a choice's branches at every role that neither sends nor receives, and declaring G *realizable* exactly when this partial function is defined everywhere — undefinedness of the merge is the static signature of a role forced to behave differently in branches it cannot distinguish. Session subtyping $\leq$ is then a second, separate coinductive relation on local types, letting a role's actual behaviour offer more external choices and fewer internal ones than its projected contract demands [13, 14, 15].

Both devices exist to answer questions about the *network as a whole* — “can every role tell the branches apart it needs to?”, “may this implementation stand in for that contract?” — while type-checking only one role's syntax at a time. Section C.2 answers the same two questions with a single judgment, *typing the whole configuration directly against* G : a choice node's rule quantifies over every branch the protocol declares and requires the *same* residual configuration to check against each one. This is the

conservative plain-merge condition, obtained without ever computing $\sqcap$ or a local type. The receiver-side subtyping direction (a receiver may accept a superset of the protocol's labels) is folded into the same rule as a label-set inclusion $I \subseteq J$. Tolerance therefore still has the same three independent knobs from Section 3 — the existential $\diamond$ (Section B.2), subtyping (now inside T-COMM, Section C.2), and the granularity of α — “flexible, tolerant of small details” is still the profile $\langle$ coarse α , $\diamond$, linear L $\rangle$; a later safety layer flips the same machinery to the universal $\square$ over permission predicates.

C The Achievability Type System

Achievability now factors into three premises, each checked by a separate terminating procedure, combined by the top rule ACH: no hallucinated capability, direct conformance of the declared skills to G , and reachability of the goal.

C.1 Capability and goal typing

$$\begin{array}{c} \text{T-EFF} \\ \frac{\Gamma(a) = \langle pre, eff \rangle \quad \varphi \models pre}{\Gamma \vdash \{\varphi\} a \{\text{sp}(\varphi, eff)\}} \end{array} \quad \begin{array}{c} \text{T-MISS} \\ \frac{a \notin \text{dom}(\Gamma)}{\Gamma \vdash a \triangleright \mathbf{X} \text{ (MISSING_CAPABILITY)}} \end{array}$$

$\text{sp}(\varphi, eff)$ is the strongest postcondition of the STRIPS effect. T-MISS fires on hallucinated planning: the plan names a tool the context does not grant, and achievability is refuted without any search.

C.2 Direct conformance typing

Skills are typed *directly* against the global protocol and the world, by the coinductive judgment $G \vdash (M; W)$ — read “ G types session M starting from world W .” There is no local type, no projection, and no separate subtyping relation: the receiver-side subtyping direction and the unobserved-choice check are structural side conditions of one rule.

$$\begin{array}{c} \text{T-END} \\ \frac{}{\text{End} \vdash (p[0]; W)} \end{array}$$

$$\begin{array}{c} \text{T-ACT} \\ \frac{\exists W'. \langle W \rangle a \langle W' \rangle \quad \forall W'. \langle W \rangle a \langle W' \rangle \implies G \vdash (p[P] \parallel \mathbb{M}; W') \quad \text{prt}(G) \cup \{p\} = \text{prt}(\mathbb{M}) \cup \{p\}}{a @ p. G \vdash (p[a.P] \parallel \mathbb{M}; W)} \end{array}$$

$$\begin{array}{c} \text{T-GOAL} \\ \frac{G \vdash (\mathbb{M}; W) \quad W \models \varphi \quad \text{prt}(G) = \text{prt}(\mathbb{M})}{\checkmark \varphi. G \vdash (\mathbb{M}; W)} \end{array}$$

$$\begin{array}{c} \text{T-COMM} \\ \frac{G_i \vdash (p[P_i] \parallel q[Q_i] \parallel \mathbb{M}; W) \quad \forall i \in I \subseteq J \quad \text{prt}(p \rightarrow q : \{\ell_i.G_i\}_{i \in I}) = \text{prt}(\mathbb{M}) \cup \{p, q\}}{p \rightarrow q : \{\ell_i.G_i\}_{i \in I} \vdash (p[q!\{\ell_i.P_i\}_{i \in I}] \parallel q[p?\{\ell_j.Q_j\}_{j \in J}] \parallel \mathbb{M}; W)} \end{array}$$

T-ACT checks the protocol and world state together with WORLD-ACT. The type $a @ p. G$ is checked before the action in W , and the remaining type G is checked after the action in W' — the same convention as G-ACT-E (Section B.2). The second premise considers every possible successor world because an Nd effect may produce different outcomes. Subject reduction therefore requires the remaining type to be valid in each successor. For capabilities without Nd , there is only one successor, so the rule is unchanged. In T-COMM the sender p offers exactly the protocol's branch set I , while the

receiver q may accept *more* ($I \subseteq J$) — the one subtyping direction the rule keeps (safe interface slack at the receiver). Requiring *every* protocol branch l_i to be checked against the *same* bystanders M is the plain-merge form of the unobserved-choice condition, so a role forced to behave differently across branches it cannot distinguish makes no derivation possible — the rule fails closed, and deadlock-freedom falls out of T-COMM itself with no merge to compute (Theorem 4). Each rule additionally carries a *participant-matching* side condition tying the protocol’s participants to the session’s:

$\text{prt}(p \rightarrow q : \{l_i, G_i\}) = \text{prt}(M) \cup \{p, q\}$ in T-COMM, $\text{prt}(G) \cup \{p\} = \text{prt}(M) \cup \{p\}$ in T-ACT, and $\text{prt}(G) = \text{prt}(M)$ in T-GOAL. These make the invariant “the roles the protocol still mentions are exactly the roles still running” hold at every node, ruling out a session that carries a stray participant the protocol has forgotten (or vice versa) — the well-formedness that lets the interleaving cases of Section D.3 match a bystander move on the session to a $\text{cap}(\cdot)$ -labelled move on the type. T-END closes the discipline at the terminus: the terminated protocol End types a single idle role $p[0]$ — and, through the congruence $p[0] \parallel M \equiv M$, any session all of whose roles have terminated — the base case of the participant invariant, since $\text{prt}(\text{End}) = \emptyset$ and an idle role contributes nothing to $\text{prt}(M)$. T-ACT is derivable only when $\langle W \rangle a \langle W' \rangle$ holds, which itself presupposes $a \in \text{dom}(\Gamma)$: this is where hallucinated capabilities die, with T-MISS as the diagnostic.

C.3 The achievability judgment

$$\frac{\text{ACH} \quad \text{caps}(G) \subseteq \text{dom}(\Gamma) \quad \exists W_0 \models \text{init}. \quad G \vdash (\text{p}_1[S_{\text{p}_1}] \parallel \dots \parallel \text{p}_n[S_{\text{p}_n}]; W_0) \wedge \Gamma; (G, W_0) \models \Diamond \varphi_{\text{goal}}}{\Gamma \vdash \{S_p\}_p : G \triangleright \Diamond \varphi_{\text{goal}}}$$

where $\{p_1, \dots, p_n\} = \text{prt}(G)$: the judgment takes one declared skill S_p per role of the protocol. Read top-to-bottom: no hallucinated tools; some plausible initial world exists from which the declared skills directly conform to the protocol (deadlock-free, every capability call guarded) *and* that same world reaches the goal. The reachability half is unchanged from Section B.2 and decided on Γ, G alone, before any S_p is known — exactly what lets the checker run on the LLM-compacted pack the moment it exists; conformance is the additional, separately checked premise once the skills are declared, and needs no transport lemma: the receiver-side subtyping slack is already inside T-COMM.

C.4 Reachability rules

The diamond is derived by the following inductive system, realized by the checker as a finite forward search over $\rightsquigarrow$; unlike T-COMM/T-ACT/T-GOAL, it never mentions a session M , which is what makes it checkable on the pack alone.

$$\begin{array}{c} \text{REACH-GOAL} \\ \frac{W \models \varphi_{\text{goal}} \quad G = \text{End} \vee \exists G'. G = \check{\varphi}_{\text{goal}}. G'}{\Gamma; (G, W) \models \Diamond \varphi_{\text{goal}}} \end{array} \quad \begin{array}{c} \text{REACH-STEP} \\ \frac{(G, W) \rightsquigarrow (G', W') \quad \Gamma; (G', W') \models \Diamond \varphi_{\text{goal}}}{\Gamma; (G, W) \models \Diamond \varphi_{\text{goal}}} \end{array}$$

$$\begin{array}{c} \text{REACH-OR} \\ \frac{\exists k \in I. \Gamma; (G_k, W) \models \Diamond \varphi_{\text{goal}}}{\Gamma; (\text{p} \rightarrow \text{q}; \{l_i, G_i\}_{i \in I}, W) \models \Diamond \varphi_{\text{goal}}} \end{array}$$

REACH-OR is derivable from REACH-STEP with G-COMM-E; we keep it because the search implements branching this way.

D Metatheory

We separate what is mechanized in Coq (Theorems 1–3, the first in relational form; Lemma 3 for a shallow embedding of the state logic; Theorems 4–5 for the single-label communication fragment with full interleaving; the head-move action case; and the concrete instances, all axiom-free) from what is proved on paper (the action-interleaving cases, the refinement of Lemma 3’s embedding to the checker’s formulas and solver, termination, and the undecidability boundary).

D.1 Soundness of the abstraction

Let C be the concrete configuration space of protocol/world pairs, A the abstract configuration space the checker explores, and $\llbracket \cdot \rrbracket : C \rightarrow A$ the abstraction. We require:

$$(\text{step-sim}) \quad C \rightsquigarrow C' \Rightarrow \text{abs}(C) \hat{\rightsquigarrow} \text{abs}(C'), \quad (\text{goal-sim}) \quad \text{goal}_{\varphi_{\text{goal}}}(C) \Rightarrow \widehat{\text{goal}}(\text{abs}(C)).$$

Lemma 1 (Transport). If $C_0 \rightsquigarrow^i C$ then $\llbracket C_0 \rrbracket \hat{\rightsquigarrow}^i \llbracket C \rrbracket$.

Proof. By induction on the reflexive-transitive closure. The empty run is reflexivity. For $C_0 \rightsquigarrow^i C' \rightsquigarrow C$, the induction hypothesis gives $\llbracket C_0 \rrbracket \hat{\rightsquigarrow}^i \llbracket C' \rrbracket$ and STEP-SIM gives $\llbracket C' \rrbracket \hat{\rightsquigarrow} \llbracket C \rrbracket$; compose them.

Mechanized as `reach_abs`, parameterized by STEP-SIM. $\square$

Lemma 2 (Reachability monotonicity). If every step of $\rightsquigarrow_1$ is a step of $\rightsquigarrow_2$ (that is, $x \rightsquigarrow_1 y$ implies $x \rightsquigarrow_2 y$), then $s \rightsquigarrow_1^i w$ implies $s \rightsquigarrow_2^i w$.

Proof. By induction on the closure $s \rightsquigarrow_1^i w$. The empty run transports by reflexivity. For $s \rightsquigarrow_1^i u \rightsquigarrow_1 w$, the induction hypothesis gives $s \rightsquigarrow_2^i u$; the hypothesis turns the last step $u \rightsquigarrow_1 w$ into $u \rightsquigarrow_2 w$; appending it gives $s \rightsquigarrow_2^i w$. Mechanized as `reach_mono`. $\square$

D.2 Refutation soundness, tolerance, monotonicity

Theorem 1 (Refutation soundness). If the abstract system has no reachable goal state from $\llbracket G, W_0 \rrbracket$, then no declared run of that pack configuration reaches φ_{goal} . Hence an *impossible* verdict is never wrong relative to the pack, provided STEP-SIM and GOAL-SIM hold.

Proof. We prove the contrapositive: if *some* declared run reaches the goal, then the abstract system reaches an abstract goal state. Suppose $C_0 \rightsquigarrow^i C$ with $\text{goal}_{\varphi_{\text{goal}}}(C)$. By Lemma 1, $\llbracket C_0 \rrbracket \hat{\rightsquigarrow}^i \llbracket C \rrbracket$, and by GOAL-SIM, $\widehat{\text{goal}}(\llbracket C \rrbracket)$. Thus $\llbracket C \rrbracket$ is an abstract goal state reachable from $\llbracket C_0 \rrbracket$, contradicting the hypothesis. Hence no declared run reaches φ_{goal} : a verdict of *impossible* — read off exactly the premise, that the abstract search exhausts without hitting an abstract goal — is never wrong. Mechanized, axiom-free as a theorem schema parameterized by the two simulation hypotheses (the existential witness is $\llbracket C \rrbracket$): $\square$

```
Theorem refutation_sound : forall s0,
  (~ exists a, reach_astype (abs s0) a /\ agoal a) ->
  (~ exists w, reach_cstep s0 w /\ cgoal w).
(* Print Assumptions refutation_sound. => Closed under the global context *)
```

Lemma 3 (The symbolic abstraction satisfies the hypotheses). Let $\llbracket G, \langle B, N \rangle \rrbracket = \llbracket G, \langle B, \psi \rangle \rrbracket$ for any accumulated constraint $\psi \in L$ with $N \neq \psi$. Let an abstract step apply the same *Add/Del* update to B and replace ψ by the strongest postcondition of the numeric effect, and let

$\widehat{goal}(G, \langle B, \psi \rangle)$ hold when $goal$ holds at control G and $\psi \wedge \varphi_{goal}$ is satisfiable under B . Then STEP-SIM and GOAL-SIM hold, and the widening $\psi \mapsto \mathcal{T}$ preserves both.

Proof. GOAL-SIM: if $\langle B, N \rangle \not\models \varphi_{goal}$ and $N \models \psi$, then N itself witnesses satisfiability of $\psi \wedge \varphi_{goal}$ under B . STEP-SIM: a concrete firing needs $\langle B, N \rangle \models pre_a$, so $\psi \wedge pre_a$ is satisfiable (N witnesses it) and the symbolic edge exists; the boolean update is identical on both sides, and the concrete successor satisfies the strongest postcondition by definition. Widening weakens ψ , which only adds abstract states and edges, so both properties are preserved (Theorem 2). Hence Theorem 1 applies to the search the implementation runs. The relational form of the abstraction and its mechanization are stated in the remark below. $\square$

Remark (shape of the abstraction, and what is mechanized). The abstraction in Lemma 3 is not a function of the concrete world: a world $\langle B, N \rangle$ is represented by $\langle B, \psi \rangle$ for every accumulated constraint ψ that N satisfies, and ψ depends on the path the search took. Theorem 1 is therefore stated and mechanized with a simulation relation $abs \subseteq C \times A$: step-sim reads “if C steps to C' and A represents C , then some A' reachable from A in one abstract step represents C' ”, goal-sim reads “if C satisfies the goal and A represents C , then A is an abstract goal state” (refutation_sound_rel; the functional form is the special case refutation_sound_fun_is_rel). Lemma 3 is then mechanized as SymbolicInstance.symbolic_refutation_sound for a shallow embedding of the state logic: constraints are predicates on numeric worlds, guards and numeric effects are relations, the strongest postcondition is the image, the widening is an arbitrary policy on edges (so the result holds for the back-edge policy the implementation uses), and the abstraction is the relation $N \models \psi$. All of this is axiom-free under Coq 8.18. What remains on paper is the refinement of that embedding to the running checker: that its quantifier-free linear-arithmetic formulas denote those predicates, and that the SMT solver decides their satisfiability correctly. An earlier version stated Lemma 3 against the functional schema, which it did not instantiate as written; this revision corrects that.

Theorem 2 (Tolerance soundness). If a *coarser* over-approximation (more edges) cannot reach the goal, neither can any finer system. Hence coarsening α to tolerate more detail never produces a false refutation.

Proof. Let $\rightsquigarrow_{fine}$ and $\rightsquigarrow_{coarse}$ be the two abstract step relations, with the coarsening an *over*-approximation: every fine step is also a coarse step, $x \rightsquigarrow_{fine} y \Rightarrow x \rightsquigarrow_{coarse} y$. Suppose, for contradiction, that the finer system reaches the goal: $s_0 \rightsquigarrow_{fine}^i a$ with $\widehat{goal}(a)$. By Lemma 2 (with $\rightsquigarrow_1 = \rightsquigarrow_{fine}$, $\rightsquigarrow_2 = \rightsquigarrow_{coarse}$), $s_0 \rightsquigarrow_{coarse}^i a$; and $\widehat{goal}(a)$ still holds. So the coarser system reaches a goal state, contradicting the hypothesis. Hence no coarser-refuted goal is reachable in any finer system: coarsening α only *adds* abstract edges, so it can never turn an unreachable goal into a reachable one — refutation is preserved. Mechanized as tolerance_sound, axiom-free. $\square$

Theorem 3 (Capability monotonicity). If $\Gamma \subseteq \Gamma'$ and Γ reaches φ_{goal} , then Γ' reaches φ_{goal} . Granting tools never makes an achievable goal impossible.

Proof. Enlarging the capability context only *adds* guarded effects: since $\Gamma \subseteq \Gamma'$, every capability firing available under Γ is available under Γ' with the same guard and effect (WORLD-ACT in B.1 quantifies over $a \in dom(\Gamma) \subseteq dom(\Gamma')$), so every step of the Γ -transition relation is a step of the Γ' -relation. Suppose Γ reaches the goal configuration C from C_0 . By Lemma 2, the same configuration is reachable under Γ' , and $goal_{\varphi_{goal}}(C)$ is unchanged because the goal predicate does not depend on Γ . Hence Γ' reaches φ_{goal} via the same witnessing run. The other premises of ACH are monotone in the same

direction $(\text{caps}(G) \subseteq \text{dom}(\Gamma) \subseteq \text{dom}(\Gamma'))$ still holds, and realizability and conformance do not mention Γ beyond $a \in \text{dom}(\Gamma)$, so the whole judgment is preserved. The transport half is mechanized as `cap_monotone`, axiom-free, stated over an inclusion of step relations; the reduction of $\Gamma \subseteq \Gamma'$ to that inclusion is the WORLD-ACT argument above, on paper. $\square$

Concrete instance. The flight skill of Section 3, variant A, is mechanized as `FlightInstance`. We prove `flight_refuted` (no run reaches *booked* $\wedge$ *confirmation*_{sent}, since no step touches *conf*) and `booking_reachable` (a booking is still reachable). Both are axiom-free, witnessing non-vacuity of Theorem 1.

D.3 Operational correspondence: subject reduction and session fidelity

The direct judgment $G \vdash (M; W)$ is not merely preserved by reduction: the session and its protocol step *in lockstep* over the labelled transition systems of Section B.2. This is the standard soundness backbone of a session calculus [2, 15], obtained here *directly*, with no projection and no merge. When actions by different participants change the world, we assume that their effects commute. Communication-only reductions satisfy this condition automatically. Goal markers require no additional condition because G-GOAL-F checks the goal in the world state where the step occurs.

Lemma 4 (Residual capability). If $(G, W) \rightsquigarrow_{\Lambda} (G', W')$, then $\Lambda \in \text{cap}(G)$.

Lemma 5 (Participant agreement). If $G \vdash (M; W)$, then $\text{prt}(G) = \text{prt}(M)$.

Both follow by induction on the transition or typing derivation, respectively; they are the capability-membership and participant invariants used in the interleaving cases below.

Theorem 4 (Subject reduction). If $G \vdash (M; W)$ and $(M, W) \xrightarrow{\Lambda} (M', W')$, then $(G, W) \rightsquigarrow_{\Lambda} (G', W')$ for some G' with $G' \vdash (M'; W')$.

Theorem 5 (Session fidelity). If $G \vdash (M; W)$ and $(G, W) \rightsquigarrow_{\Lambda} (G', W')$, then $(M, W) \xrightarrow{\Lambda} (M', W')$ for some M' and W' with $G' \vdash (M'; W')$.

We prove both by induction on the typing derivation $G \vdash (M; W)$, with a case analysis on the transition; the one auxiliary fact used throughout is that typing respects pointwise-equal sessions (`ctypes_ext`), which lets the interleaving cases reassociate independent updates.

Proof of Theorem 4 (subject reduction). By induction on $G \vdash (M; W)$.

Case T-END ($G = \text{End}$, every role idle). Then M has no output or pending action, so no session transition exists and the claim holds vacuously.

Case T-GOAL ($G = \checkmark \varphi . G_0$, with $W \not\models \varphi$ and $G_0 \vdash (M; W)$, $\text{prt}(G_0) = \text{prt}(M)$). A marker is not a session construct, so the step $(M, W) \xrightarrow{\Lambda} (M', W')$ is a step of G_0 's session; the induction hypothesis gives $(G_0, W) \rightsquigarrow_{\Lambda} (G', W')$ with $G' \vdash (M'; W')$. Since $W \not\models \varphi$, rule G-GOAL-F discharges the marker and performs the same step, $(\checkmark \varphi . G_0, W) \rightsquigarrow_{\Lambda} (G', W')$; the residual typing is the one the induction hypothesis provides. No stability hypothesis is needed: the marker is checked at W , where the step happens. *Case T-COMM*, with head $p \rightarrow q$ and branches G_i . Here M is $p[q! \dots] \parallel q[p? \dots] \parallel M_0$, and each branch satisfies $G_i \vdash (p[P_i] \parallel q[Q_i] \parallel M_0; W)$. The transition is a communication $\Lambda = r \mid t$.

Because p offers only outputs to q and q only inputs from p , the sender/receiver shapes force a dichotomy.

- *Head move* ($r = p$). Then $t = q$ and $l = l_k \in I$, and $M' = p[P_k] // q[Q_k] // M_0$. Rule G-COMM-E gives $(G, W) \rightsquigarrow_{p l_k q} (G_k, W)$, and $G_k \vdash (M'; W)$ is exactly the k -th premise.
- *Interleaving move* ($r \neq p$). The shapes then force $r, t \notin \{p, q\}$: r is an output so $r \neq q$, and t is an input so $t \neq p$, while $t = q$ would force $r = p$. Since r, t lie in the bystanders M_0 , the same communication is enabled in each branch's configuration $p[P_i] // q[Q_i] // M_0$ (the head updates do not touch r, t). The induction hypothesis at each i yields G_i' with $(G_i, W) \rightsquigarrow_{\Lambda} (G_i', W)$ and $G_i' \vdash (p[P_i] // q[Q_i] // M_0'; W)$, where M_0' is M_0 after the bystander step. As $\text{prt}(\Lambda) = \{r, t\}$ is disjoint from $\{p, q\}$, rule G-COMM-I assembles $(G, W) \rightsquigarrow_{\Lambda} (p \rightarrow q: \{l_i. G_i'\}, W)$, and re-applying T-COMM (the sender/receiver at p, q are untouched, and each branch is retyped by the induction hypothesis, reassociating the independent updates via `ctypes_ext`) gives the residual typing.

Communications leave the world fixed, so no independence hypothesis is needed. For a world-changing action at the head (T-ACT), G-ACT-E matches and the residual G is typed at the same post-effect world the rule reaches — whichever successor world the session's firing picks, the universal premise of T-ACT types the residual at it. Interleaving *two* actions requires their effects to commute ($\text{eff}_a \circ \text{eff}_b = \text{eff}_b \circ \text{eff}_a$ for disjoint a, b), a frame-independence side condition that participant-disjointness alone does not supply; under it the T-ACT interleaving case goes through identically. $\square$

Proof of Theorem 5 (session fidelity). By induction on $G \vdash (M; W)$, inverting the global step $(G, W) \rightsquigarrow_{\Lambda} (G', W')$. T-END admits no global step in the correspondence, so the claim holds vacuously. For T-GOAL, the step is G-GOAL-F; its premise $(G_0, W) \rightsquigarrow_{\Lambda} (G', W')$ is realised by the induction hypothesis as a session step, which is the required one — the marker exists only on the type side and is discharged by the same transition. For T-COMM, the global step is either G-COMM-E — and the prescribed head communication $p l_k q$ is enabled by S-COMM, landing in the k -th premise — or G-COMM-I with $\text{prt}(\Lambda) \cap \{p, q\} = \emptyset$; then the induction hypothesis on any branch produces a bystander communication, whose endpoints lie outside $\{p, q\}$, so S-COMM fires it on M and T-COMM retypes the result. The action cases mirror those of Theorem 4. $\square$

Mechanization. Both theorems are mechanized axiom-free for the single-label communication fragment — the fragment in which bystander interleaving is unconditional — in `DirectTypingSR.v`, over labelled session and global transition systems, with no projection, merge, or local type:

```
Theorem subject_reduction : forall W G s s' L,
  ctypes W G s -> lstep L s s' ->
  exists G', gstep W L G G' /\ ctypes W G' s'.
Theorem session_fidelity : forall W G G' s L,
  ctypes W G s -> gstep W L G G' ->
  exists s', lstep L s s' /\ ctypes W G' s'.
(* Print Assumptions => Closed under the global context (both). *)
```

The head-move case for world-changing *actions* is additionally mechanized in `DirectTyping.v` (`type_directed_safety`, also `progress`). Goal labels appear in neither label alphabet: the

paper’s correspondence uses the same fused goal rule (G-GOAL-F) as the Coq `GS_Goal`. Participant-matching side conditions, labelled action interleaving, and multi-branch choice are still absent from the mechanization. Completing a faithful mechanization of the paper rules remains work in Section 5.4.

Concrete instance. `HandoffInstance` mechanizes the planner/worker handoff of Section 1 on both sides. The *good* pairing — planner sends, worker delivers, goal checked — is proved `ctypes`-typed (`good_typed`) and its run is proved to reach a world satisfying the goal (`good_runs_to_goal`). The *bad* pairing — both roles start with an input, exactly the deadlock of Section 1 — is proved `Stuck` (`bad_stuck`), and the contrapositive of `progress` then gives, for free, that *no non-trivial global type can ever type it* (`bad_untypable`): the rule rejects the classical deadlocking handoff without any projection ever being computed.

D.4 Incompleteness, by design

Proposition 1 (Incompleteness). $\Gamma ; G \not\models \Diamond_{init} \varphi_{goal}$ may hold while no concrete runtime execution reaches φ_{goal} : the witnessing abstract path can be spurious when α collapses a distinction that mattered. The system is *sound for and incomplete for*.

This is the price of decidable, tolerant over-approximation; tightening α trades tolerance for precision. The residue is confined to two edges, motivating the layered architecture:

- **Payload faithfulness** (bottom edge): a tool’s declared post-condition (“filters to fares ≤ 500 ”) may be false — a runtime obligation for a deterministic monitor (Layer C), not a static one.
- **Intent fidelity** (top edge): the formalized goal may not capture what the user meant — irreducibly semantic, surfaced at the single compaction checkpoint for human review.

D.5 Decidability and the autonomy boundary

Theorem 6 (Termination and soundness of the widened search). In the fragment with finitely many roles (static topology), regular global types represented by finite tail-recursive control graphs, decidable state logic L , and L -definable effects, the widened symbolic search terminates on every pack. It decides the widened abstract judgment; for $\Gamma ; G \not\models \Diamond_{init} \varphi_{goal}$ its IMPOSSIBLE answer is sound (Theorem 1 with Lemma 3) and its ACHIEVABLE answer may be spurious (Proposition 1).

Proof. The pack-level achievability judgment $\Gamma ; G \not\models \Diamond_{init} \varphi_{goal}$ (Section B.2) is an existential reachability query over configurations (G', W) ; we exhibit the terminating procedure and place its two answers.

The search space is finite. A configuration has two components. (i) *Control*, the residual global type G' reachable from G by $\rightsquigarrow$. Since G is a regular tree (tail-recursion: finitely many distinct subtrees, Section A.3) and each $\rightsquigarrow$ step either descends into a subtree (G-COMM-E, G-ACT-E) or erases a marker (G-GOAL-E), the set $\{G' : \rightsquigarrow\}$ is a subset of the finitely many subtrees of G , hence finite. (ii) *Symbolic world*. The boolean part is a valuation $B : Pred \rightarrow B$ over the finitely many predicates named in the pack, so there are $2^{|Pred|}$ boolean states. The numeric part is summarized by an accumulated constraint $\psi \in L$ (a conjunction of the guards taken and the Nd -postconditions assumed); on a control-graph back edge the reference procedure applies the widening $\psi \mapsto \top$, forgetting accumulated numeric constraints. This one-element numeric summary is an over-approximation and, together with the finite boolean valuations, leaves only finitely many symbolic worlds. The product with finite control is therefore finite.

Each edge is computable. Firing a capability requires checking $W \models pre_a$ and forming sp ; testing goal-satisfaction requires $W \models \varphi$. Both are entailment/satisfiability queries in quantifier-free linear integer

arithmetic with uninterpreted predicates — a decidable theory, discharged by an SMT solver. Branch selection (REACH-OR) is a finite disjunction. Existence of an initial world is decided by the same SMT query over *init*; each satisfying symbolic initial valuation seeds the finite search.

Termination and correctness. Forward search (breadth-first over $\rightsquigarrow$) maintains a visited set of (control, symbolic-world) pairs; because that set is finite it adds each pair at most once, so the search halts. It answers ACHIEVABLE iff it enumerates a goal-satisfying configuration, and IMPOSSIBLE otherwise. Soundness of the IMPOSSIBLE verdict is Theorem 1 with Lemma 3; and widening only *enlarges* the reachable set (it weakens constraints, adding edges), so by Theorem 2 it never introduces a false refutation. Hence the search always terminates and decides the widened abstract judgment; IMPOSSIBLE transfers to the exact judgment, ACHIEVABLE does not (Proposition 1). $\square$

Why not exact decidability. With unbounded integers, updates $x := x \pm 1$, and guards $x = 0, x \geq 1$ under recursive control, packs encode two-counter machines, so the exact judgment $\Gamma; G \models \Diamond_{init} \varphi_{goal}$ is undecidable even with a fixed set of participants. The widening is what buys termination; the price is the spurious-ACHIEVABLE residue of Proposition 1.

Proposition 2 (Undecidability in a dynamic-topology extension). Consider the extension G^{+ii} of the grammar with *add r . G*, which creates (adds) a new participant during execution: the new participant runs a declared finite behaviour template and brings one fresh numeric variable, initialized by the step that adds it. With an unbounded role population and dynamic channels, achievability in G^{+ii} is undecidable — and no widening restores termination, since the reachable control space itself becomes infinite.

Proof sketch. By reduction from the control-state reachability problem for communicating finite-state machines (CFSMs), which is undecidable [16]. Fix a CFSM instance: finite-control machines M_1, M_2 exchanging messages over unbounded FIFO channels, and a target control state q_f of M_1 ; deciding whether a run reaches q_f is undecidable.

We construct a skill whose goal is achievable iff the CFSM reaches q_f . Each machine M_i becomes a role whose process follows its control graph. When a machine sends message m , the process adds a new participant carrying m . When a machine receives m , it synchronizes with the oldest outstanding participant for that channel. This unbounded set of participants represents the unbounded FIFO queue. The message order is enforced using sequence numbers: each new participant stores the current send counter, and a receive is allowed only when that number matches the receive counter. This requires a fresh variable for each participant, which is available in G^{+ii} but not in the base fragment with a fixed *Var*. The *Asg/Nd* machinery of Section A.2 supplies the counter updates. A boolean predicate at_i is established by the single capability guarding M_1 's entry into q_f , and we set $\varphi_{goal} = at_i$. By construction a run of the skill reaches a state satisfying φ_{goal} iff the CFSM has a run reaching q_f : the courier discipline then puts the skill's reachable configurations in correspondence with the CFSM's channel contents and control states. The construction is a computable map from CFSM instances to packs, so a decision procedure for achievability would decide CFSM control-state reachability — impossible. Hence achievability is undecidable once new participants may be added during execution. (The finiteness argument of Theorem 6 breaks at exactly the point this reduction exploits: an unbounded number of couriers makes the set of reachable controls infinite.) $\square$

The two results identify a concrete boundary: when the set of participants is finite and fixed, the widened search terminates and its *impossible* verdict is sound. If an unbounded number of new participants can be added during execution, no such finite search can decide the problem in

general. This result concerns only the number of participants; it does not mean that all forms of agent autonomy are undecidable. The implementation returns UNKNOWN when a pack violates the static-topology side condition of Theorem 6, unless an independent capability or establisher refutation has already proved it impossible.

D.6 The partition of obligations

Corollary 1. The architecture separates “will the goal be achieved” into four obligations on disjoint substrates:

Concern	Where	Mechanism
goal achievability	static, terminating	Coq specification; deterministic search; no LLM
per-step safety / least privilege	Layer B (future)	same engine, $\square$ + permission tiers
payload faithfulness	Layer C (future), runtime	deterministic monitors with blame
intent fidelity	top edge, synthesis	one human checkpoint

D.7 Verifiable feedback beyond a scalar reward

The verdict is a heterogeneous verification signal rather than a scalar score. An IMPOSSIBLE result identifies which premise failed and carries a blocking frontier; ACHIEVABLE carries a witness path and explicitly retains the two deferred obligations; UNKNOWN is an abstention outside the decidable fragment, not a refutation. These structured outcomes can serve as cheap negative labels for automated skill or prompt search without an LLM judge, while the intent-fidelity checkpoint keeps the irreducible semantic decision with a human. We treat such search as an application of the verifier, not as a contribution evaluated here.

D.8 Which result covers each verdict

The implementation returns IMPOSSIBLE under five reason codes and abstains under a sixth. They do not all rest on the same result, and a review of an earlier version rightly asked which are covered by Theorem 1. The table states, for each code, what the verdict refutes and which result, machine-checked or on paper, covers it.

Reason code	What it refutes	Covering result	Machine-checked?
MISSING_CAPABILITY	The protocol invokes a tool absent from Γ ; no run of the declared protocol can fire it.	T-Miss: no T-Act derivation exists; decided without search.	The rule; no mechanization needed.
GOAL_UNSAT (establisher closure)	No capability in Γ adds some goal atom: refutes every protocol over Γ , not only the declared one.	A protocol-independent instance of Theorem 1 (empty step set for that atom).	Theorem 1 schema mechanized; the instance is on paper.
GOAL_UNSAT, BLOCKED_GUARD (search)	No run of the declared protocol reaches the goal; a mandatory guard never holds.	Theorem 1 with Lemma 3 (the symbolic abstraction satisfies step-sim and goal-sim); Theorem 2 for the widening.	Theorems 1–2 mechanized, in relational form; Lemma 3 mechanized for a shallow embedding of the state logic; the formula semantics and the SMT solver are trusted (D.2).

Reason code	What it refutes	Covering result	Machine-checked?
NON_PROJECTABLE	A role must act in a branch it cannot distinguish: the declared protocol deadlocks or hands off wrongly.	T-Comm fails closed; Theorem 4 (typed sessions do not reach this deadlock).	Single-label fragment mechanized; multi-branch choice on paper.
NON_CONFORMANT	A declared behaviour does not refine its role, or the whole-session judgment has no derivation from the witness's initial world: the verdict on G does not transport to the declared skills.	Side condition of Ach; T-Comm/T-Act/T-Goal.	Not mechanized; the algorithmic adapter (projection, merge, conformance) is asserted equivalent to the declarative judgment.
DYNAMIC_TOPOLOGY (UNKNOWN)	Nothing; the pack spawns participants, outside the decidable fragment.	Proposition 2.	On paper.

The two protocol codes refute the declared workflow, not the goal. A protocol with one conforming branch that reaches the goal and one non-conforming branch fails T-Comm, which requires every branch to conform, even though a goal-reaching execution exists. Theorem 1 does not cover this case and the implementation does not claim it does: the reason codes are distinct, and under the goal-only policy (Section 3.1) a protocol rejection becomes UNKNOWN with reason PROTOCOL_ONLY unless the protocol-independent establisher refutation has already fired. Under the default policy the verdict is a statement about the workflow as written, which is what a host that will execute that workflow needs; a rejection there is an instruction to repair the protocol, not proof that the goal is unreachable by other means.

E Token economics of pre-execution refutation

Historical assumptions and current recomputation.

The illustrative estimates below retain the source-era compaction prompt assumptions. Recomputing the same corpus with the current implementation on 28 September 2026 gives 13,016 modeled compaction tokens for the seven refutations, rather than 12,239; the modeled runtime total remains 1,067,913, giving $82.0 \times$ rather than $87 \times$ leverage. For the eight non-refuted packs, the current preparation estimate is 14,900 tokens against 476,280 modeled runtime tokens, or 3.1% rather than 2.9%. These changes reflect prompt length, not new measured savings. The historical table and ratios below must not be read as current measurements.

The practical case for deciding achievability *before* execution is an economic one, so we state it in tokens rather than adjectives.

Two asymmetries carry it. First, no model sits in the trusted path: the schema gate, direct conformance and Z3 reachability spend **zero tokens**, and the deterministic front-end spends zero as well.

Within preparation and checking, only optional LLM compaction spends model tokens. Its output can be reused for an unchanged skill and compatible task conditions; changed declarations require renewed review and checking, and may require new compaction. Any model calls during admitted agent execution are additional SkillC-on tokens.

Second, an agent turn re-sends its whole context: writing S for the harness prompt, K for the loaded skill, g for the mean tokens appended per turn and o for output per turn, a run of T turns reads $T(S+K)+gT(T-1)/2$ input and oT output tokens. Compaction is linear in the skill and paid once; a doomed run is *quadratic* in its turns and recurs on every invocation.

This appendix models counterfactual runtime waste rather than measuring a SkillC-on/off comparison. The separate experiments in Appendix F actually run the ungated baseline and count its tokens. The estimates below describe a prevented run under assumed turn counts, not new provider-usage observations.

Each refutation reason carries a (low, typical, high) band for how long an agent flails before that failure stops it, and how many agents are billed. With conservative defaults ($S=2500$, $g=700$, $o=250$) and a median real consumer skill ($K \approx 1.7K$ tokens), compaction costs $\sim 2.8K$ tokens against the following avoided waste per prevented invocation:

Verdict reason	turns (lo–typ–hi)	waste/run	leverage
MISSING_CAPABILITY	3–8–14, 1 agent	50 240	18 ×
GOAL_UNSAT	8–18–30, 1 agent	176 040	63 ×
NON_CONFORMANT	6–15–30, 2 agents	261 900	94 ×
NON_PROJECTABLE	6–15–30, 2 agents	261 900	94 ×
BLOCKED_GUARD	12–25–50, 1 agent	305 750	110 ×

The ordering is the robust part, and it is the expected one. `MISSING_CAPABILITY` is cheapest at run time: the agent calls a tool that is not there and learns immediately, and most of the cost is the improvisation that follows. `BLOCKED_GUARD` is dearest, because a precondition no run can satisfy is precisely the case an agent cannot learn from — it retries to the harness turn cap. The two multi-party reasons bill two contexts. `GOAL_UNSAT` additionally carries a cost the token bill does not show: a run whose goal conjunct has no establisher can terminate *believing* it succeeded. `DYNAMIC_TOPOLOGY` claims no savings at all, because an abstention prevents nothing.

Under these modeling assumptions, the seven structural refutations in the 15-spec corpus are assigned 12.2K compaction tokens and $\sim 1.07M$ avoided runtime tokens (band 0.28M–3.15M). The $\sim 87 \times$ ratio compares modeled avoided runtime with modeled preparation cost, not measured end-to-end SkillC-off and SkillC-on totals. The modeled break-even falls within the first prevented run (0.9%–5.5% of its estimated token cost); this is not a general empirical guarantee. Deterministic preparation adds no model tokens, but subsequent admitted execution can.

The honest denominator for a healthy skill, where the check buys nothing and still costs something, is 2.9% of one successful run over the corpus and 4.0% for a median real skill; with the deterministic front-end, zero. Prompt caching changes the price of the wasted tokens, not their number. Halving every waste default leaves leverage at $9 \times$ – $55 \times$ and the ordering unchanged. The model, its defaults and their justification are in the artifact (`skillc.tokens`, `skillc.cost`); the Coq development costs no tokens and is amortized over every skill ever checked.

F Supporting Measurements

These tables support the results in Section 5. They are not additional task populations and should not be pooled with the original formal corpus or historical estimates.

F.1 Live task families

Each row contains four conditions. “Gate + agent” uses deterministic preparation and the same runtime agent as the baseline. “LLM + agent” adds language-model preparation. “Plan” is the separate direct executor, whose extraction failure prevents an equal-success comparison. CSV means comma-separated data; SQL is the language used for database queries; MCP is the Model Context Protocol for connecting tools; LangExtract is the text-extraction library used in one family.

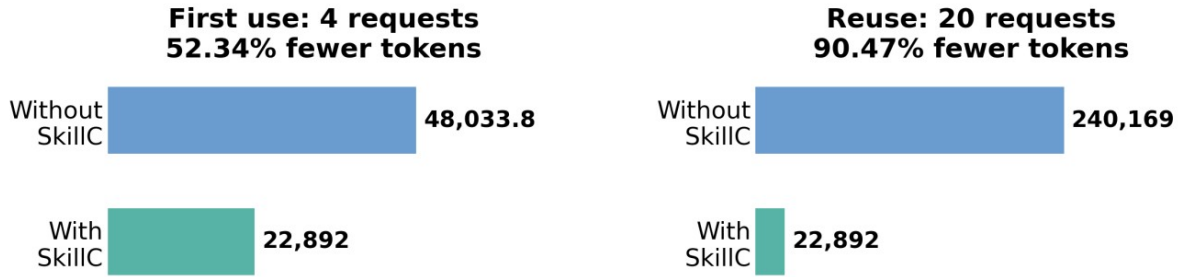
Task (4 profiles)	Ungated	Gate + agent	LLM + agent	Plan
CSV sales	1,888	1,258	4,041	
SQLite orders	2,251	1,262	4,042	
Git diff	2,982	1,252	4,032	
ZIP manifest	2,197	1,248	4,020	
Thumbnail	2,192	1,258	4,031	
HTTP catalog	3,105	1,276	4,066	
Web-app testing	2,809	1,380	5,926	
Frontend rendering	2,517	1,422	7,502	
GIF creation	2,920	1,440	8,196	
Algorithmic art	2,548	1,428	12,530	
MCP roundtrip	2,510	1,408	8,245	
Dataset Viewer	2,645	1,490	6,570	
Static review	2,991	1,440	7,118	
LangExtract	1,665	1,196	4,015	
Total	35,220	18,758	84,334	

The first three configurations complete both achievable cases in every family. The direct executor completes only one of the two achievable extraction cases. Every checked configuration rejects the 28 impossible scenarios in this bounded comparison.

F.2 Repeated requests with simulated tools

The four task names identify the evaluated public skill examples. Preparation is paid once; the baseline column sums five runtime trials per task.

Independently impossible task	With SkillC: once	Without: 5 trials	Outcomes
mcp-builder	4,338	94,542	0/5 successes
frontend-design	4,039	59,791	0/5 successes
slack-gif-creator	8,737	12,182	0/5 successes
algorithmic-art	5,778	73,654	0/5 successes
Total	22,892	240,169	0/20 successes



Real model calls; simulated tool effects. Preparation paid once per unchanged task.
First use uses trial means; reuse counts all 20 measured attempts.

Figure 5. The same measured preparation cost spread over repeated requests for unchanged impossible tasks. Tool effects were simulated.

The first-use baseline is 48,033.8 tokens, the sum of the four five-trial means. The fifth evaluated skill, web-app testing, is excluded because its goal can be achieved by another legal route and two ungated trials succeeded. Across all five skills, 92 fresh model calls used 260,242 tokens. The four-task savings claim does not apply to that mixed set.

F.3 Historical results, kept separate

An earlier experiment contains 134 agent executions. Of 66 executions in configurations rejected by the checker, 20 nevertheless produced verified results on small inputs. The remaining 46 failures used 15,841,106 measured runtime tokens. Selecting failures after the fact is not evidence that blocking all 66 would preserve completion.

Historical subset	With SkillC	Without SkillC	Reduction
PDF: compaction once, 4 failures	22,011	157,454	86.02%
XLSX: compaction once, 4 failures	32,028	209,766	84.73%
PDF + XLSX: 8 failures	54,039	367,220	85.28%
PDF + XLSX: first-use means	54,039	91,805	41.14%
All 46 failures: mixed evidence	209,195	15,841,106	98.68%

The eight PDF/spreadsheet failures have measured costs on both sides and reuse one preparation per skill. The 98.68% historical figure instead combines 54,039 measured and 155,156 estimated preparation tokens; it is not interchangeable with the 46.74% live result. The earlier pilot also remains negative evidence: 20,064 preparation tokens exceeded 1,919 credited avoidable tokens, and three of six achievable cases were falsely rejected.

F.4 Evidence and reproduction

The public repository linked in Section 6 contains the task definitions, reviewed reference outcomes, provider usage, saved prompts and instructions for reproducing the experiments. The live evidence reconciles 227 main provider calls and 24 calls from the corrected extraction experiment. Primary usage is 138,564 tokens across the four configurations. A retained diagnostic run adds 7,563 tokens, making 146,127 in total; diagnostics are not added to the primary baseline. Historical measurements are pinned to commit dbb855d1e51c550ce7cdabaf4624e0c2de0325ec.

The separate 32-scenario prompt-development comparison is not held-out evidence: its revised prompt was evaluated on the same scenarios after errors were observed. The controlled-English A/B on 150 skills (Section 5.5) and the blind-execution benchmark on 490 pairs (Sections 4.1 and 5.5–5.6) are

separate evidence sets with their own frozen plans; they are not pooled with the headline token savings, and the token counts they report use a tokenizer proxy rather than provider usage records. Their data, executor reports and scoring scripts are kept on the repository's data branch.

G Submission Coverage and Reading Notes

The supplied technical-talk template asks for a conceptual explanation rather than implementation file listings. The main proposal therefore contains the essential method, model purpose, evaluation design, results and limits; this paper adds depth and the complete proofs.

Requirement	Where it is addressed
Business purpose, contribution and prior work	Main proposal Introduction and Related Work; full paper Sections 1–2.
Conceptual method, architecture, AI output and unexpected input	Main Methodology; full paper Section 3 and workflow figure; Controlled English in Section 3.6.
Theory development and proofs	Main Methodology refers to this supplement; full paper Section 3 and Appendices A–D, with verdict coverage in D.8.
Training, loss, splits and learning curves	Explicitly not applicable to checker training; pretrained model and prompt-development limits in Section 3; the small requirement classifier of Section 5.6 is a side study.
Data, selection, examples, governance and reproducibility	Main Data; full paper Section 4, the benchmark design in 4.1, and the repository evidence link.
Baseline, metrics, costs, scale, strengths and limits	Main Results; full paper Section 5 and Appendix F; the direct-verdict baseline and negative results in Sections 5.5–5.6.
Implementation status and audience guidance	Main integration section and conclusion; full paper Sections 6 and 8, including the runtime monitor and pre-session hook.
Author responsibility, originality, rights and release	Full paper Section 7; final author approval required before upload.

Reading the original formal appendices.

Appendix A defines the symbols and building blocks. Appendix B defines permitted steps. Appendix C states the rules used to judge a described task. Appendix D proves the stated results and identifies where simplified descriptions or unbounded systems limit them. Appendix E is a cost model, not another measured benchmark. Formal notation is retained rather than paraphrased into a different theorem.

Theorem 2 has the intended direction: if even an enlarged set of possible behaviors cannot reach the goal, the smaller system cannot either. Its final proof sentence must not be read as forbidding new, potentially spurious routes when behaviors are added. Proposition 2 concerns unrestricted growth in participants; broadening numerical conditions alone does not bound that growth. It is not a claim that every coarse analysis must fail. The implementation's coordination procedure and its correspondence to the mathematical rules are not fully machine-proved. These qualifications accompany the original proof wording.

Before upload.

Replace the integration-repository placeholder with the intended accessible link, and state the exact Claude model version used by the compaction, execution and adjudication sub-agents of the real-skill benchmark (Section 4.1), taken from the run logs. Authors must confirm accuracy, originality, rights, release permission, and readiness to explain the work. If this continues an earlier MLADS submission,

the submission must also identify what is new. Automatic-decline timing is an organizer process, not evidence that these confirmations can be deferred.